

Anemoi profiling - interactive

Cathal O'Brien

Why profile?

- Enable you to run the largest models possible
- Ensure your program is running as fast as possible
- Understand bottlenecks
 - Memory usage
 - Dataloading
 - Compute time
- Identify performance “gotchas”
 - Excessive data movement
 - Memory leaks

Memory profiling

- Problem: Maximise memory use without hitting the limit
 - More performant
 - More efficient
- Probably the most common performance issue when designing new experiments
- Avoid the dreaded CUDA OutOfMemory error

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 1.41 GiB. GPU 2 has a total capacity of 39.56 GiB of which 870.75 MiB is free.
```

- Why did this happen?
- Where is the GPU memory being used?

Highest level – peak memory usage

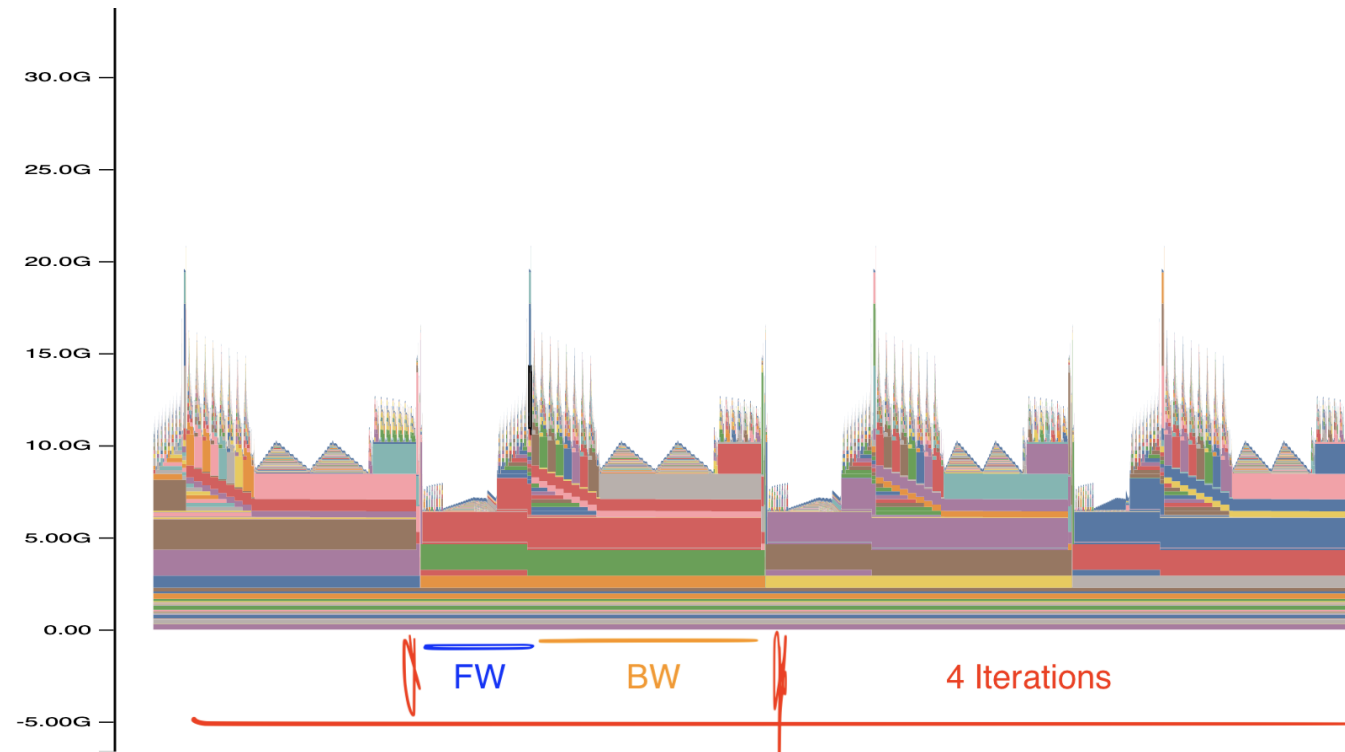
- Memory usage is impacted by a variety of factors
 - Model parameter count
 - Batch size
 - Input resolution
- First thing you should do is check the peak memory usage

```
PyTorch CUDA memory summary, device ID 0
```

CUDA OOMs: 0		cudaMalloc retries: 0		
Metric	Cur Usage	Peak Usage	Tot Alloc	Tot Freed
Allocated memory	12732 MiB	62755 MiB	623712 GiB	623699 GiB

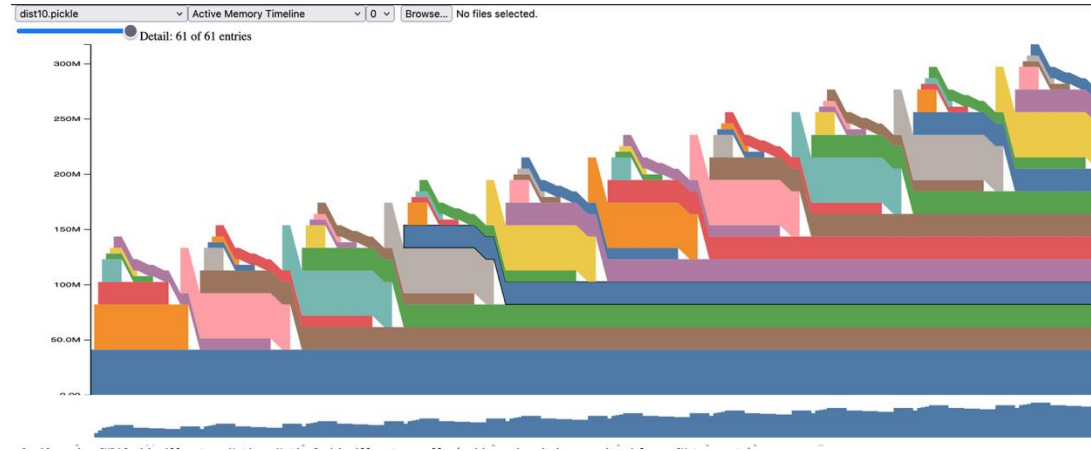
- But that doesn't tell the whole story...

Memory snapshots

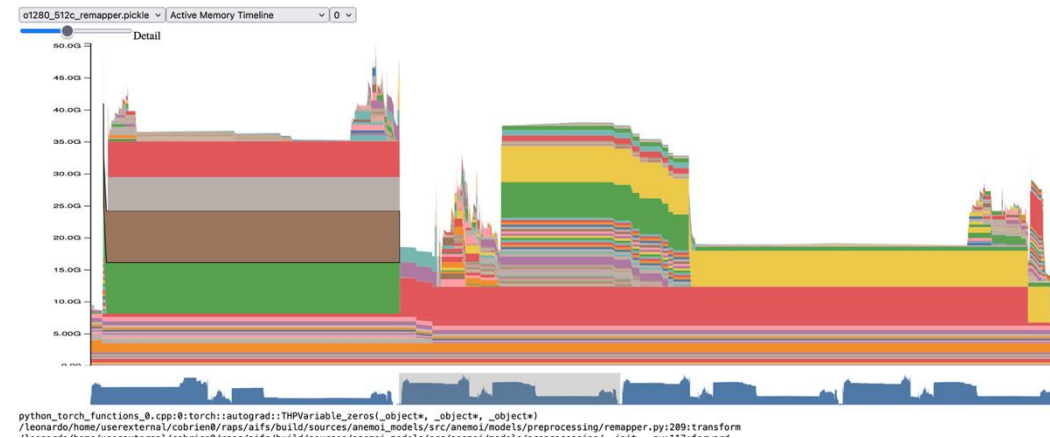


- Memory usage varies over time
 - Peak memory usage of 20GB vs average of ~12GB

- Memory leak



- Large blocks of memory



How to view a memory snapshot

- Add an entry to your config ->
- Find the output file

```
ls $HOME/anemoi/outputs/profiler/*
```

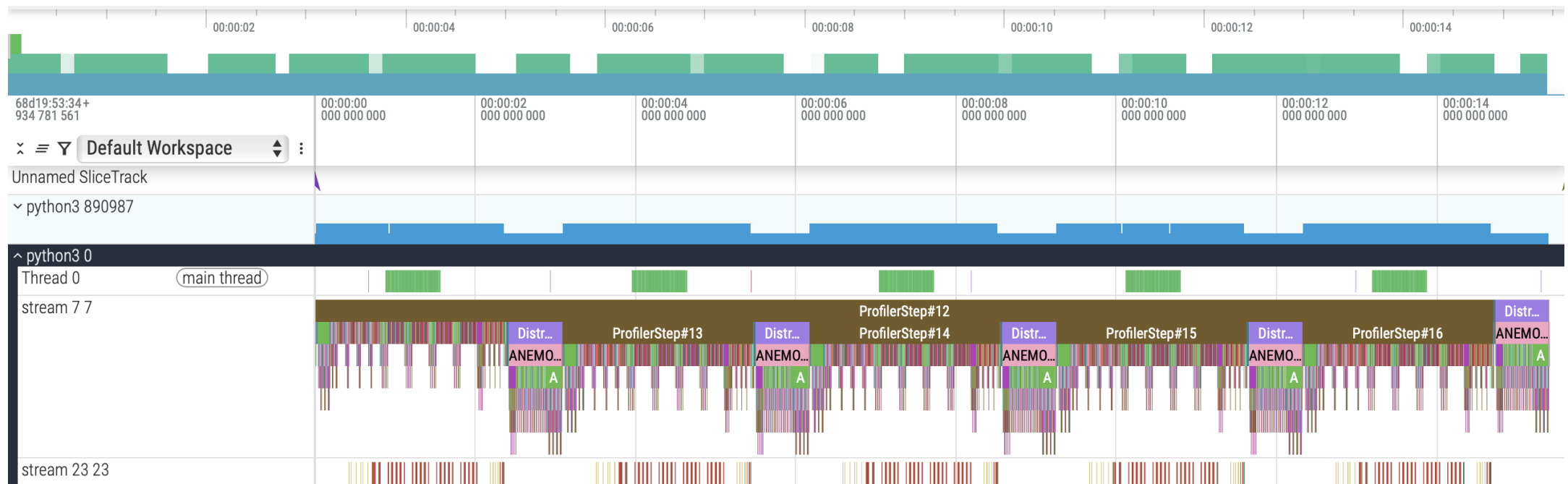
```
profiler/d62851ba-ab68-465c-b093-e6bc9de2fefcd:  
memory_snapshot.pickle
```

- Upload to
https://docs.pytorch.org/memory_viz

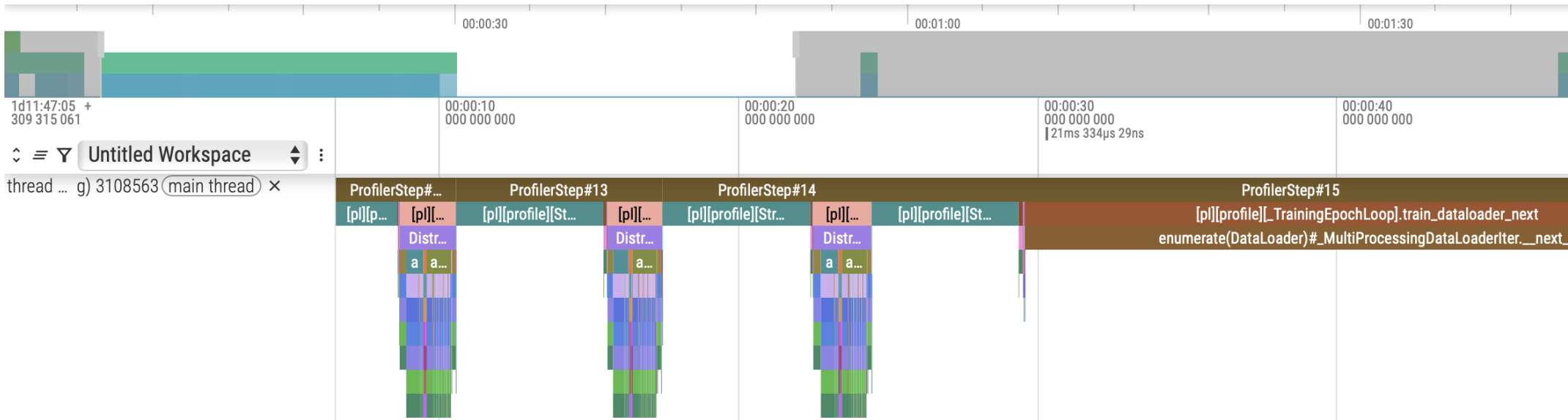
```
defaults:  
- data: zarr  
- dataloader: native_grid  
- diagnostics: evaluation  
- datamodule: single  
- hardware: example  
- graph: multi_scale  
- model: gnn  
- training: default  
- _self_  
  
config_validation: False  
  
# Add the following lines to enable memory snapshots  
diagnostics:  
  benchmark_profiler:  
    snapshot:  
      enabled: True
```

Runtime profiling

- You might wonder
 - Where is the GPU spending its time while running?
 - Is it going as fast as it possibly could?
- Examining traces can help understand a GPU's runtime



Dataloader bound



Exercises

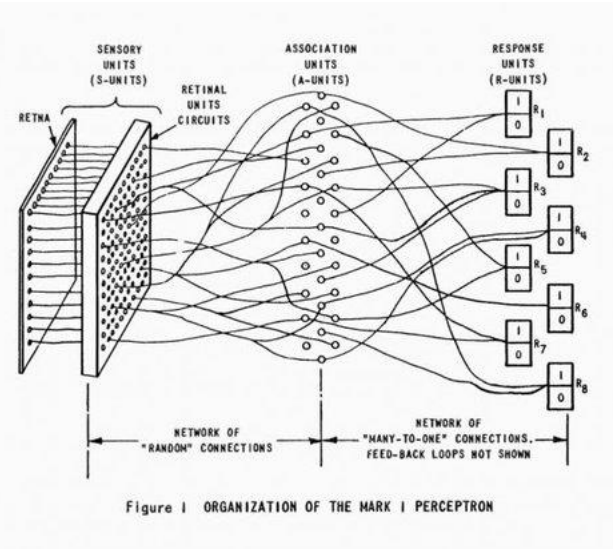
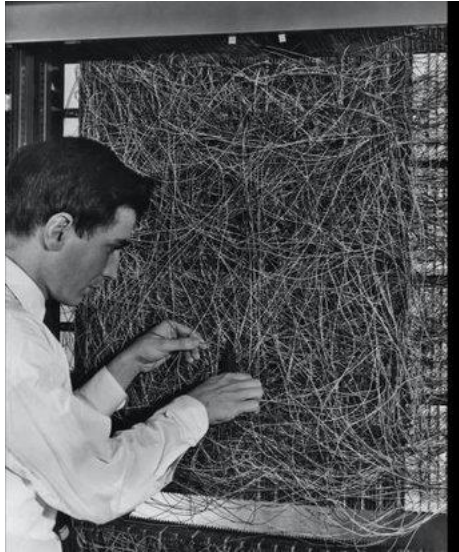
1. Get something running
2. Memory profiling
 1. Print peak memory usage
 2. Play with batch size, #channels to see it change
 3. Generate a memory snapshot
 4. See if you can find some lines in the code where most memory is allocated
3. Runtime profiling
 1. Generate a runtime trace and open it in Perfetto
 2. Change the number of workers, can you see a change in the trace?

Effective data loading

Cathal O'Brien

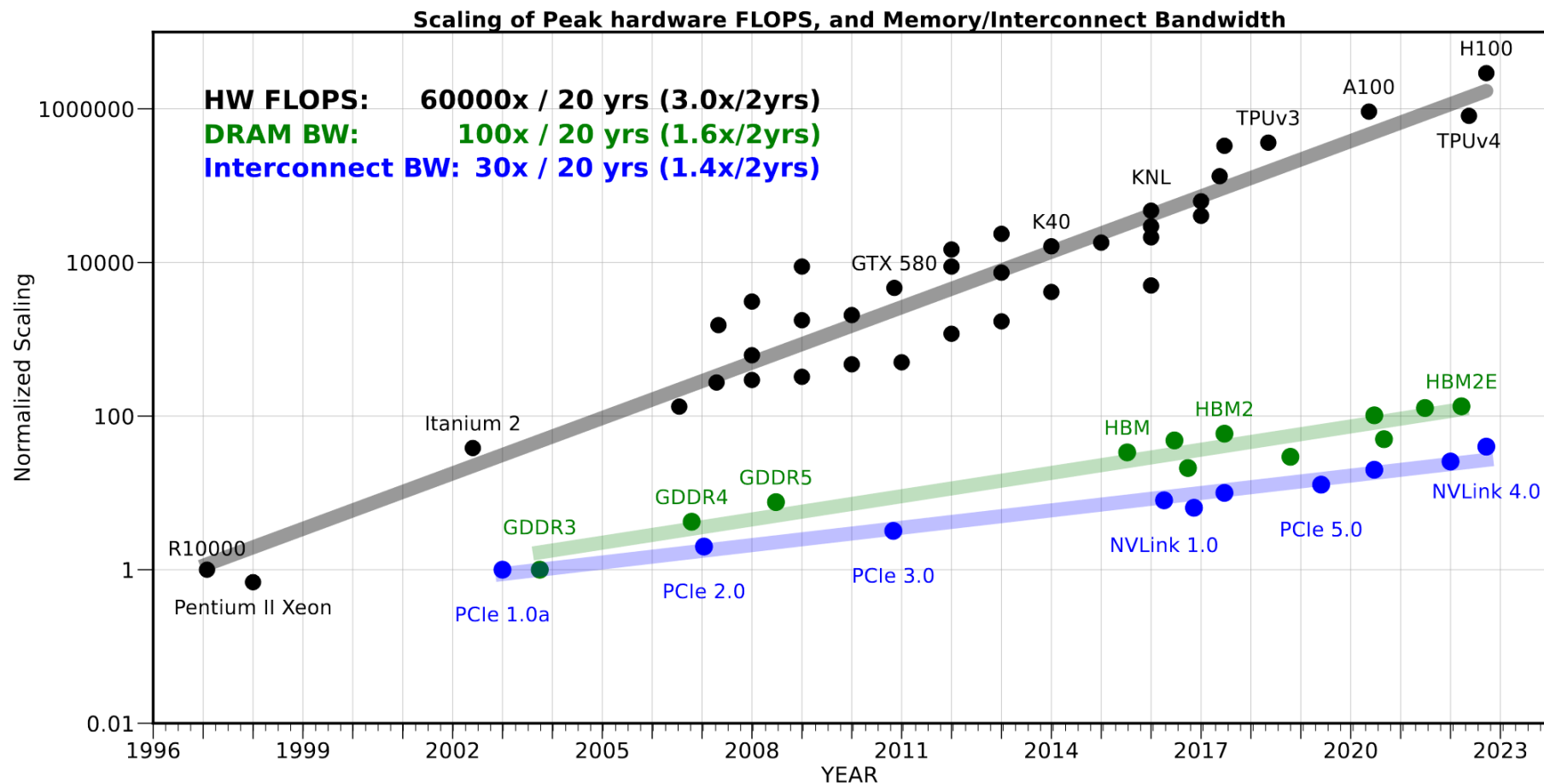
Background

- Machine learning is an old idea

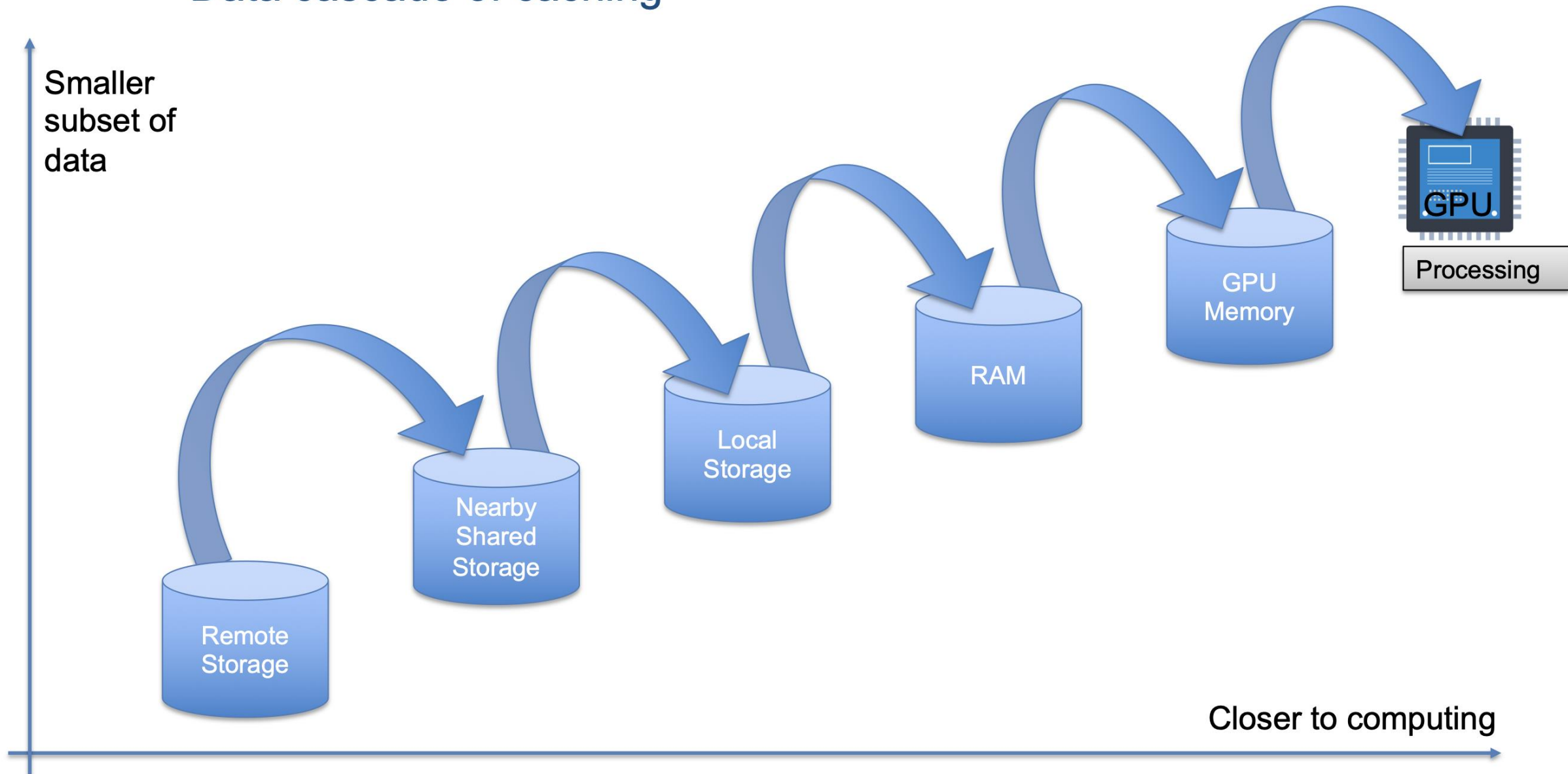


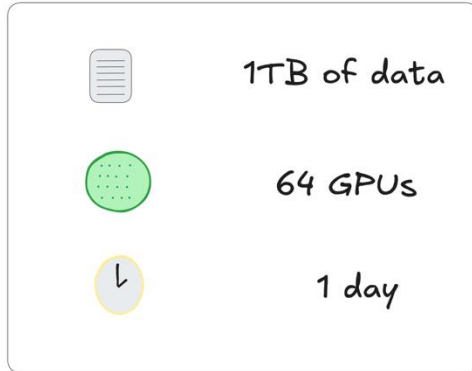
Frank Rosenblatt
programming his
Perceptron, 1970s

- Increased compute power makes machine learning possible today
 - Moores law
 - "Domain specific compute architectures" e.g. GPUs
- But there is a problem...as compute got faster, other components couldn't keep up



Data cascade of caching



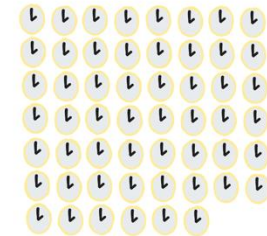
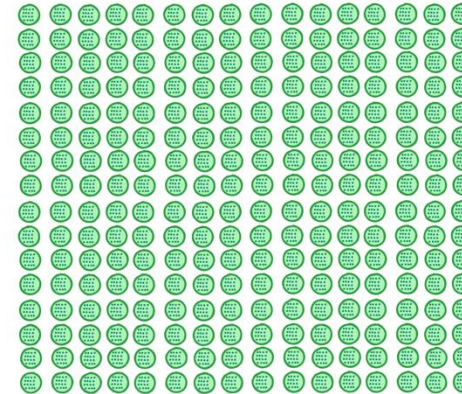
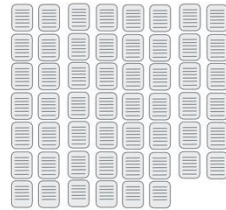


Input data

Compute

Walltime

Llama 3





1TB of data



64 GPUs



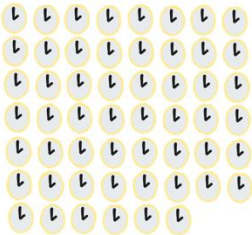
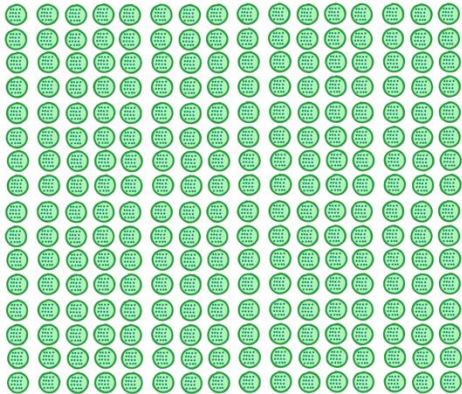
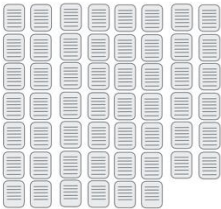
1 day

Input data

Compute

Walltime

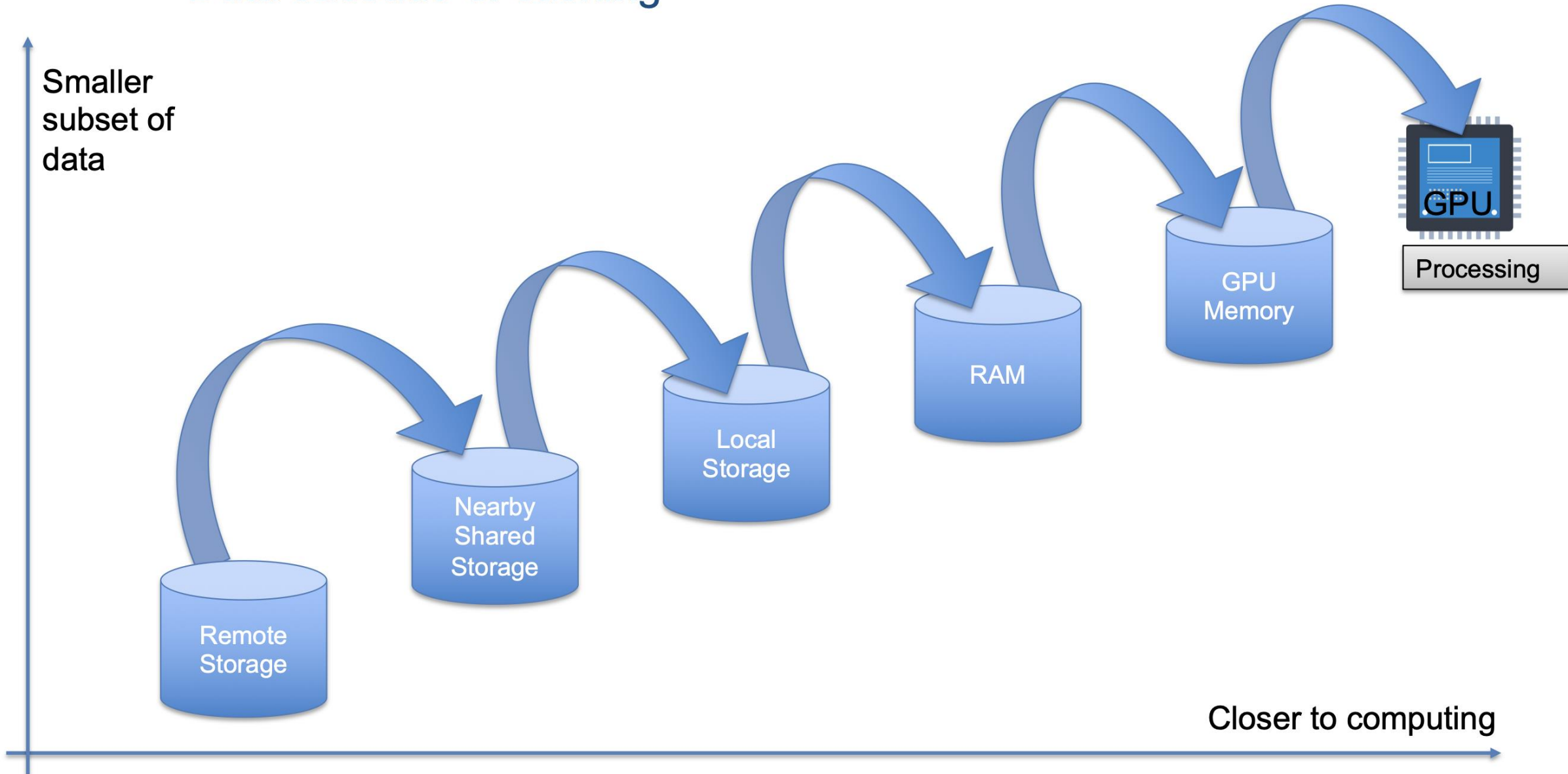
Llama 3



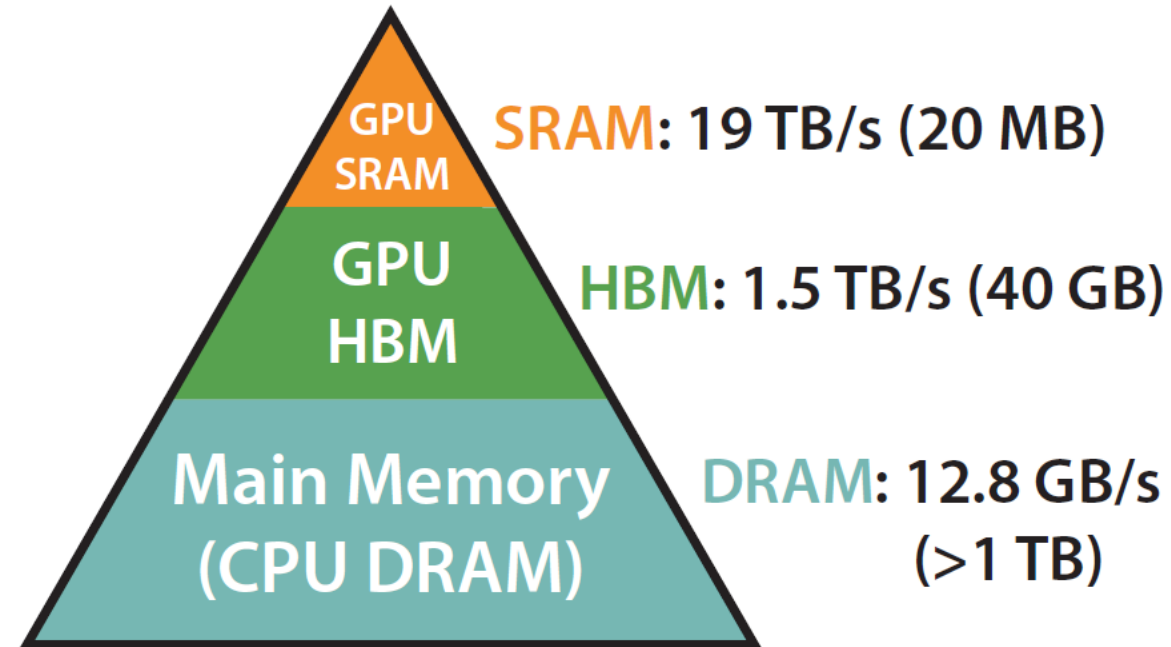
AIFS single v1



Data cascade of caching

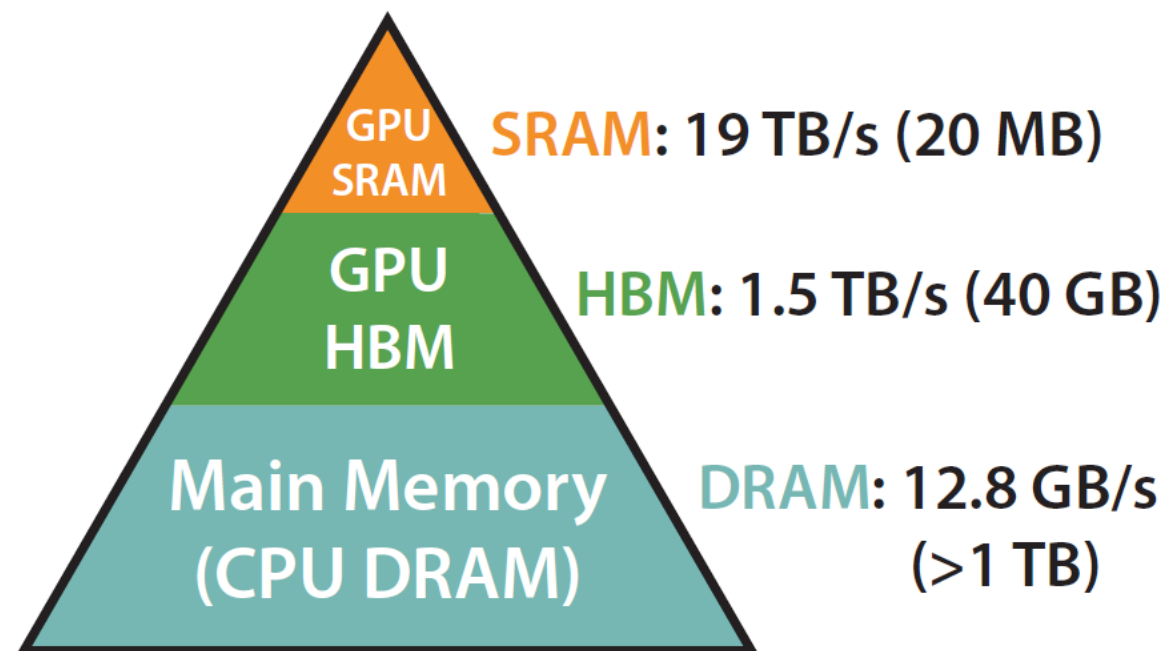


GPU Memory Hierarchy



Memory Hierarchy with
Bandwidth & Memory Size

GPU Memory Hierarchy

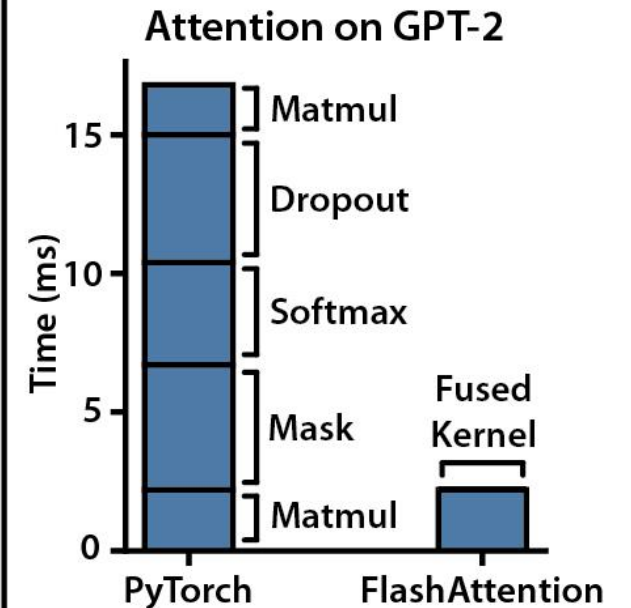
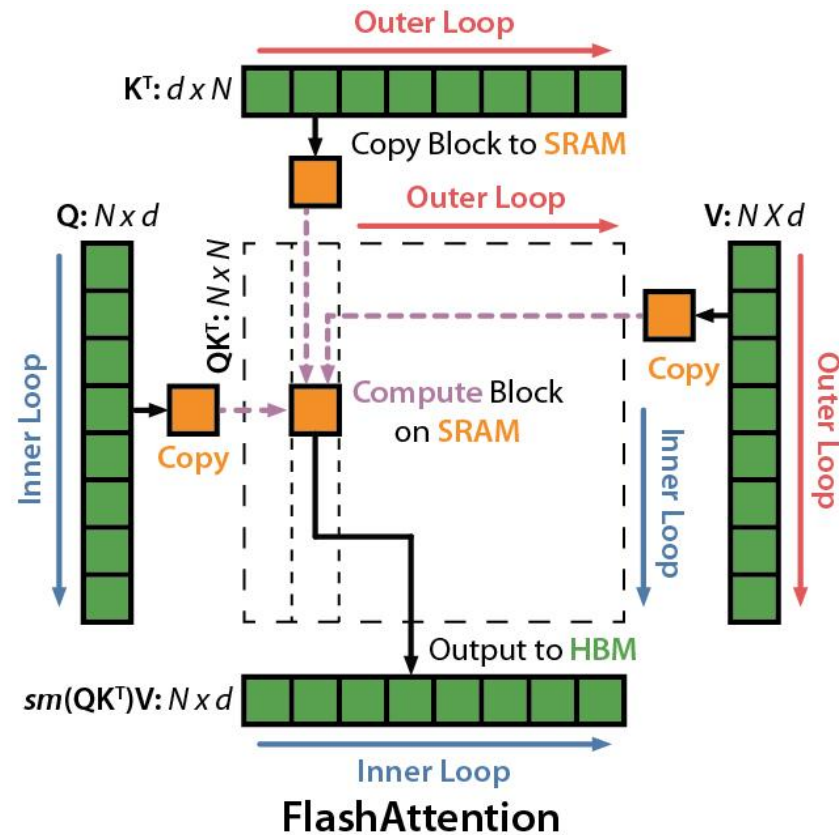
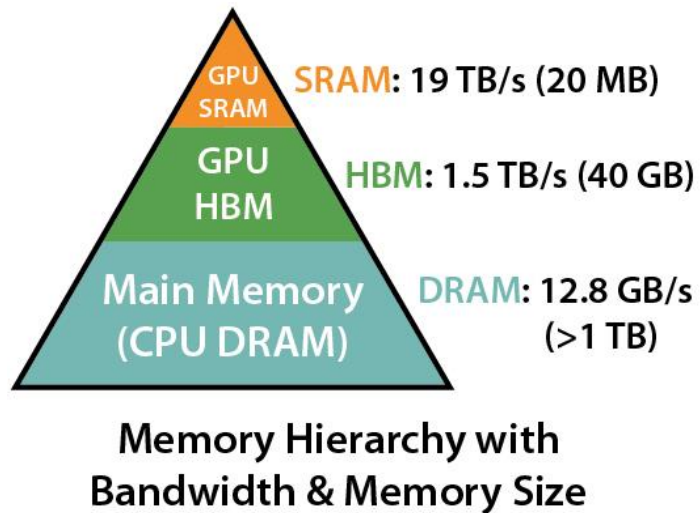


Memory Hierarchy with
Bandwidth & Memory Size



Memory hierarchy in action

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Torch.compile

- “torch.compile” can improve your model's performance
 - Optimises code by fusing kernels
 - Can't compile all code e.g. checkpointing
 - JIT compilation

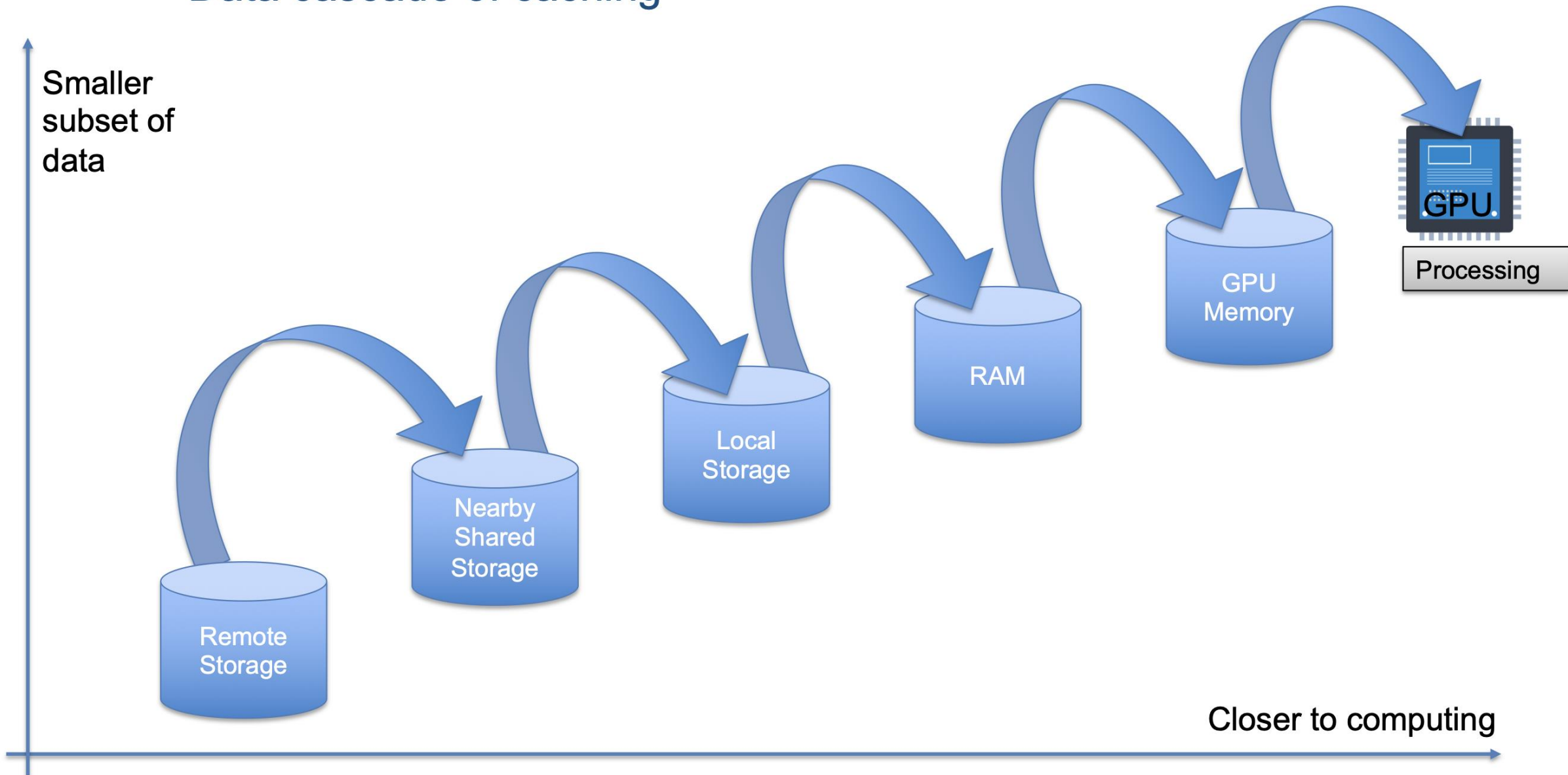
```
from torchvision.models import densenet121

model = densenet121().to(torch.float16).cuda()
model_opt = torch.compile(model)
```

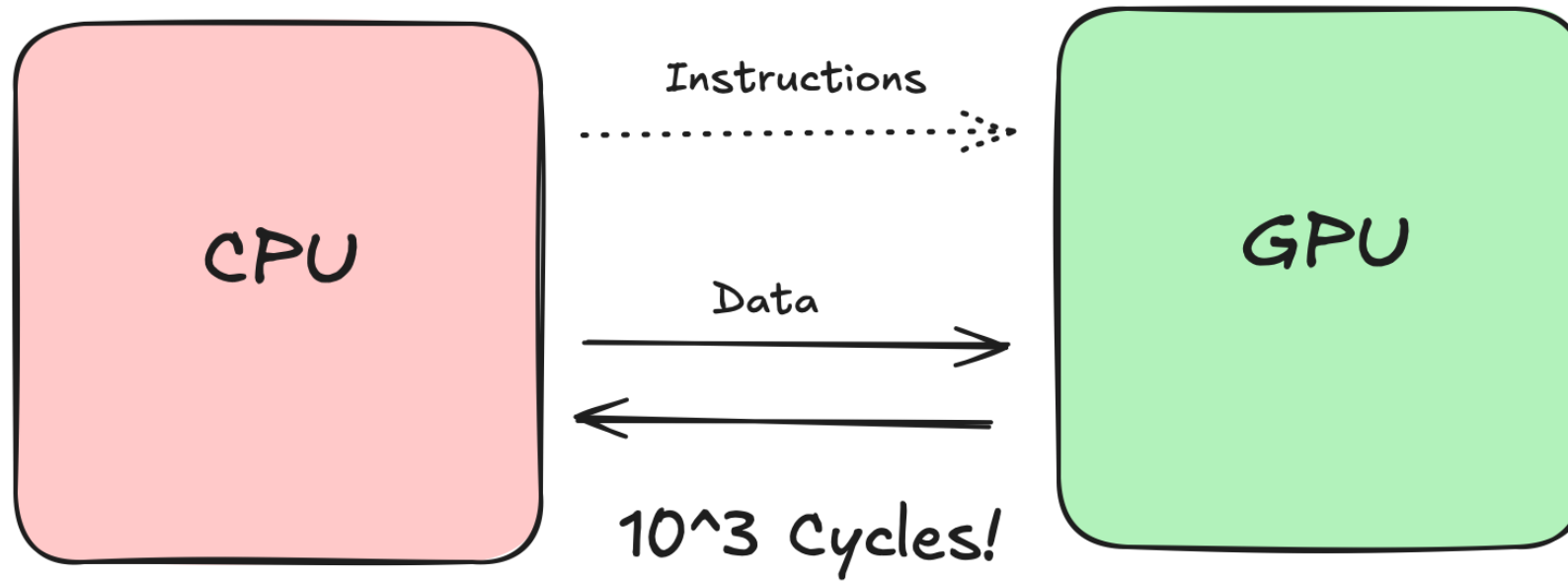
Torch.compile

```
eager eval time 0: 0.46534725952148437
eager eval time 1: 0.21482131958007813
eager eval time 2: 0.24442906188964844
eager eval time 3: 0.24335157775878907
eager eval time 4: 0.2497589111328125
eager eval time 5: 0.25520912170410154
eager eval time 6: 0.25265242004394534
eager eval time 7: 0.2440663604736328
eager eval time 8: 0.2254874267578125
eager eval time 9: 0.2381739501953125
~~~~~
compile eval time 0: 103.4196640625
compile eval time 1: 0.7687618408203125
compile eval time 2: 0.004799583911895752
compile eval time 3: 0.0048024640083312985
compile eval time 4: 0.003957216024398804
compile eval time 5: 0.003933631896972657
compile eval time 6: 0.15991769409179687
compile eval time 7: 0.003874687910079956
compile eval time 8: 0.0038679680824279783
compile eval time 9: 0.0038515520095825195
~~~~~
(eval) eager median: 0.24424771118164063, compile median: 0.0043783999681472775, speedup: 55.784695998203695x
~~~~~
```

Data cascade of caching



What is a CPU?

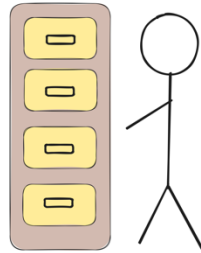


- Must be driven by a CPU
 - CPU launches all programs (called ‘kernels’) on GPU
 - CPU and GPU have separate memory spaces and must explicitly transfer data (slow)

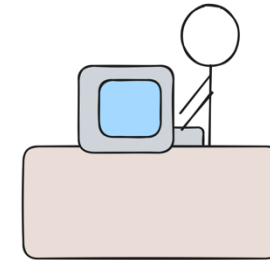
CPU

- Primary use in machine learning is to prepare data for the GPU

Chris

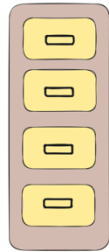


Gina

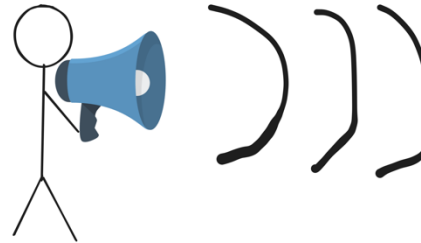


CPU

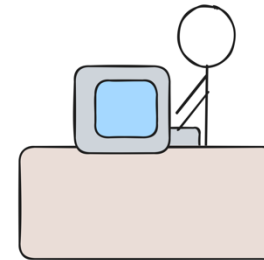
- Primary use in machine learning is to prepare data for the GPU



Chris



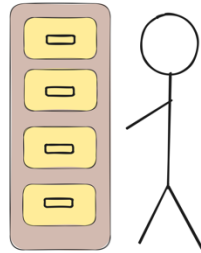
Gina



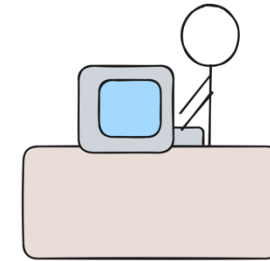
CPU

- Primary use in machine learning is to prepare data for the GPU

Chris



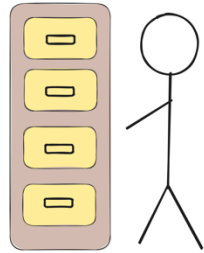
Gina



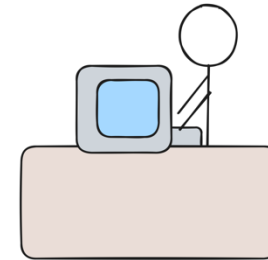
CPU

- Primary use in machine learning is to prepare data for the GPU

Chris



Gina

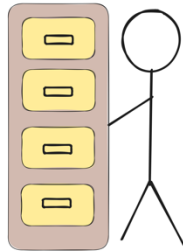


```
dataloader:  
  num_workers:  
    training: 0
```

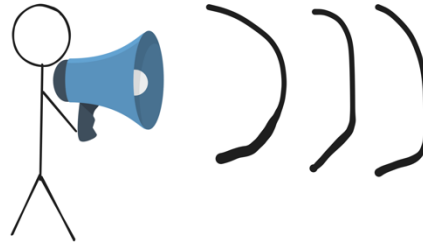
CPU

- Primary use in machine learning is to prepare data for the GPU

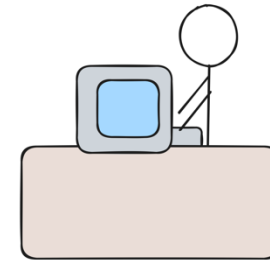
Dana



Chris

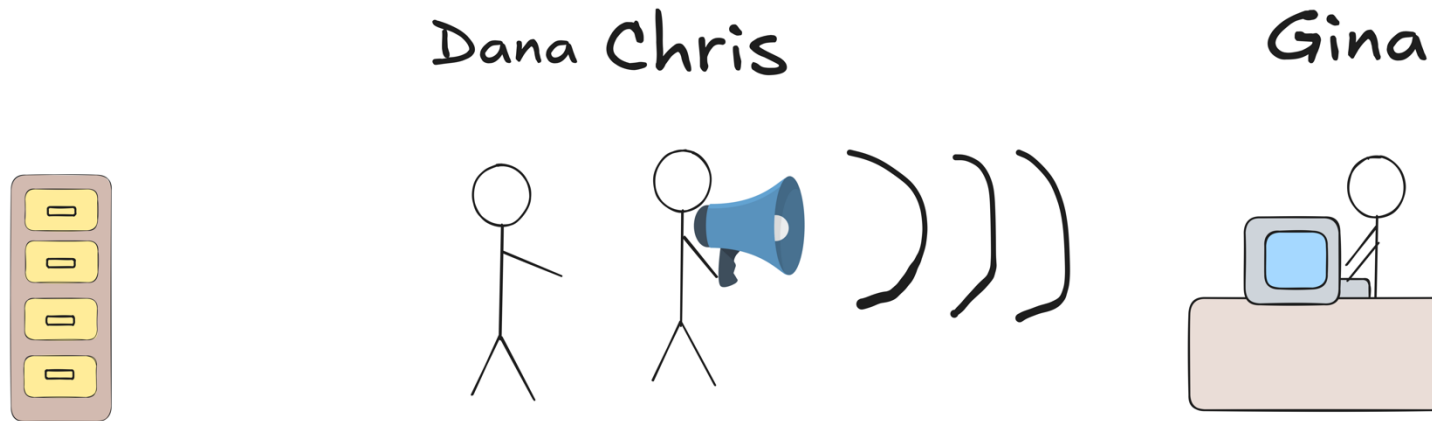


Gina



CPU

- Primary use in machine learning is to prepare data for the GPU



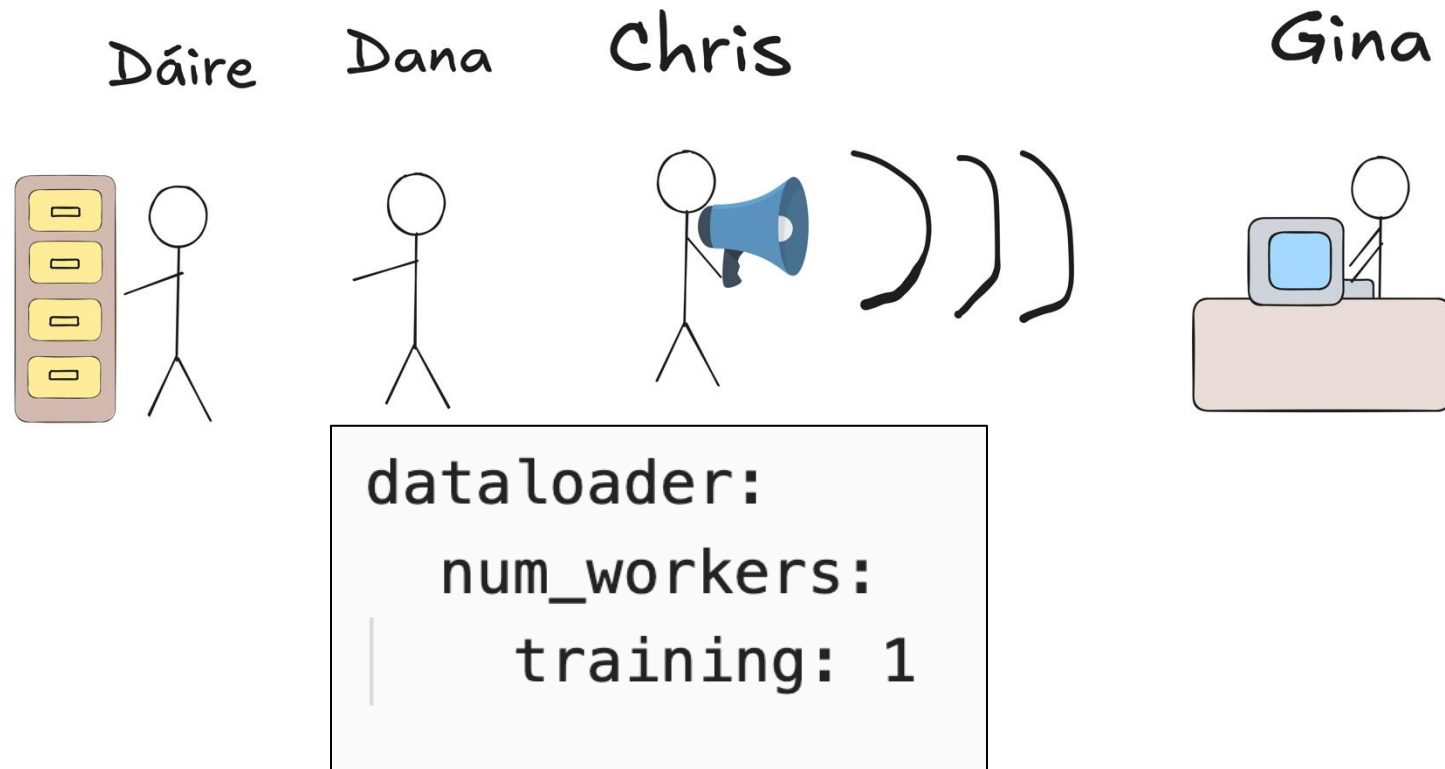
CPU

- Primary use in machine learning is to prepare data for the GPU



CPU

- Primary use in machine learning is to prepare data for the GPU



Know your access patterns : transposition

Example data

9 fields

14 dates

Real data

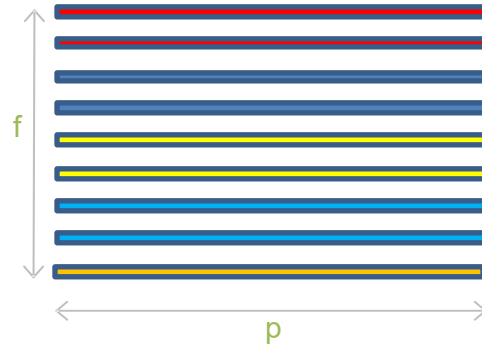
100 fields

O(10k) dates

O(1M) grid

dimensions

Data saved by field

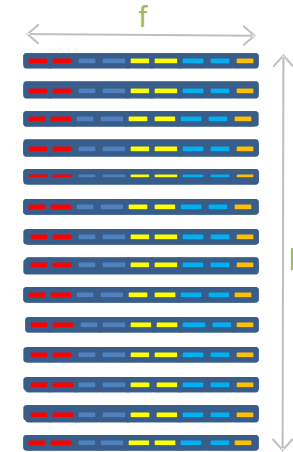


One file contains the
timeseries of one given **field**

9 fields

≠

Data saved by date



One file contains the set of
values for one given **date**

9 dates

Know your access patterns : transposition

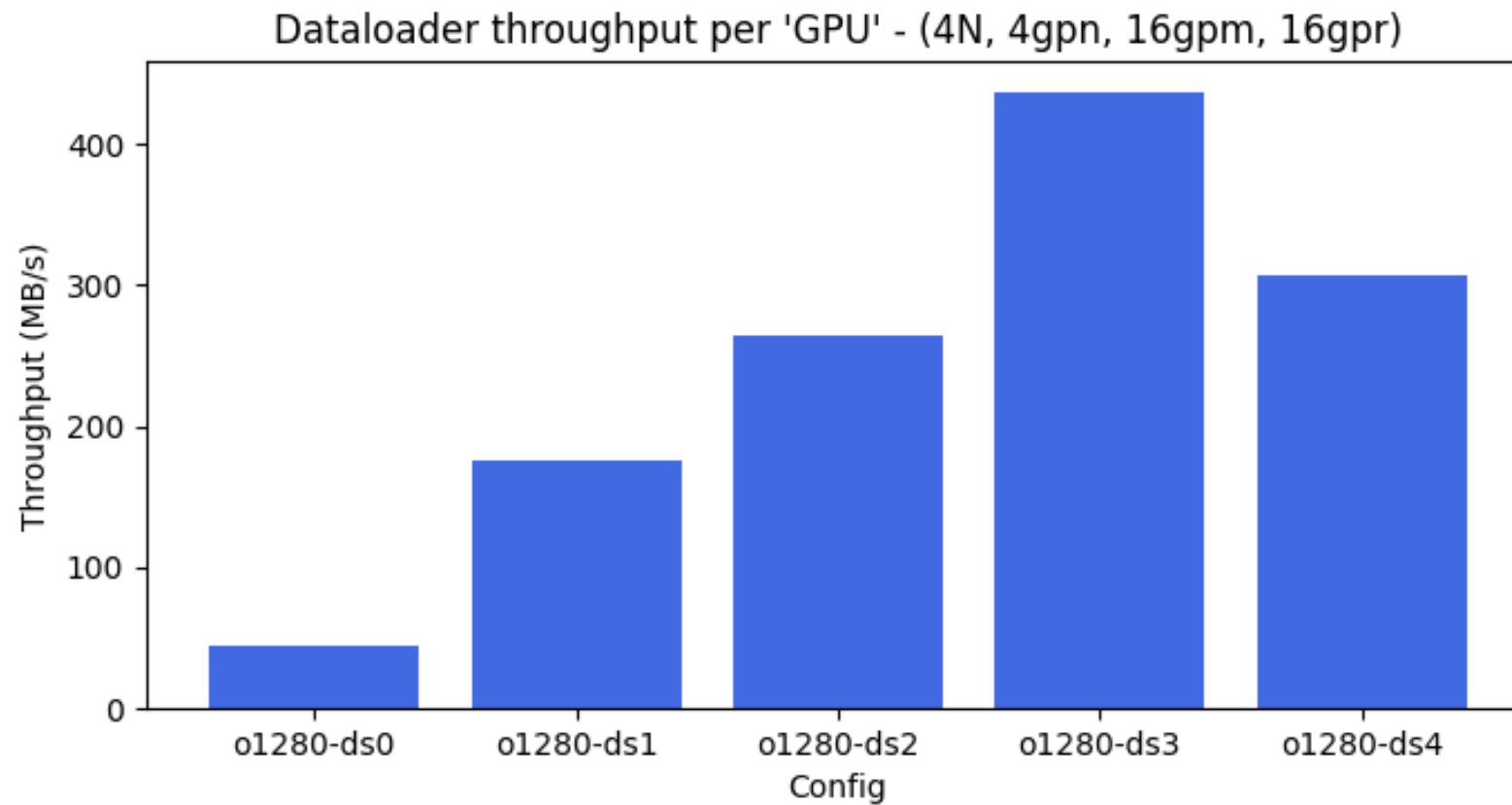


Know your access patterns : transposition

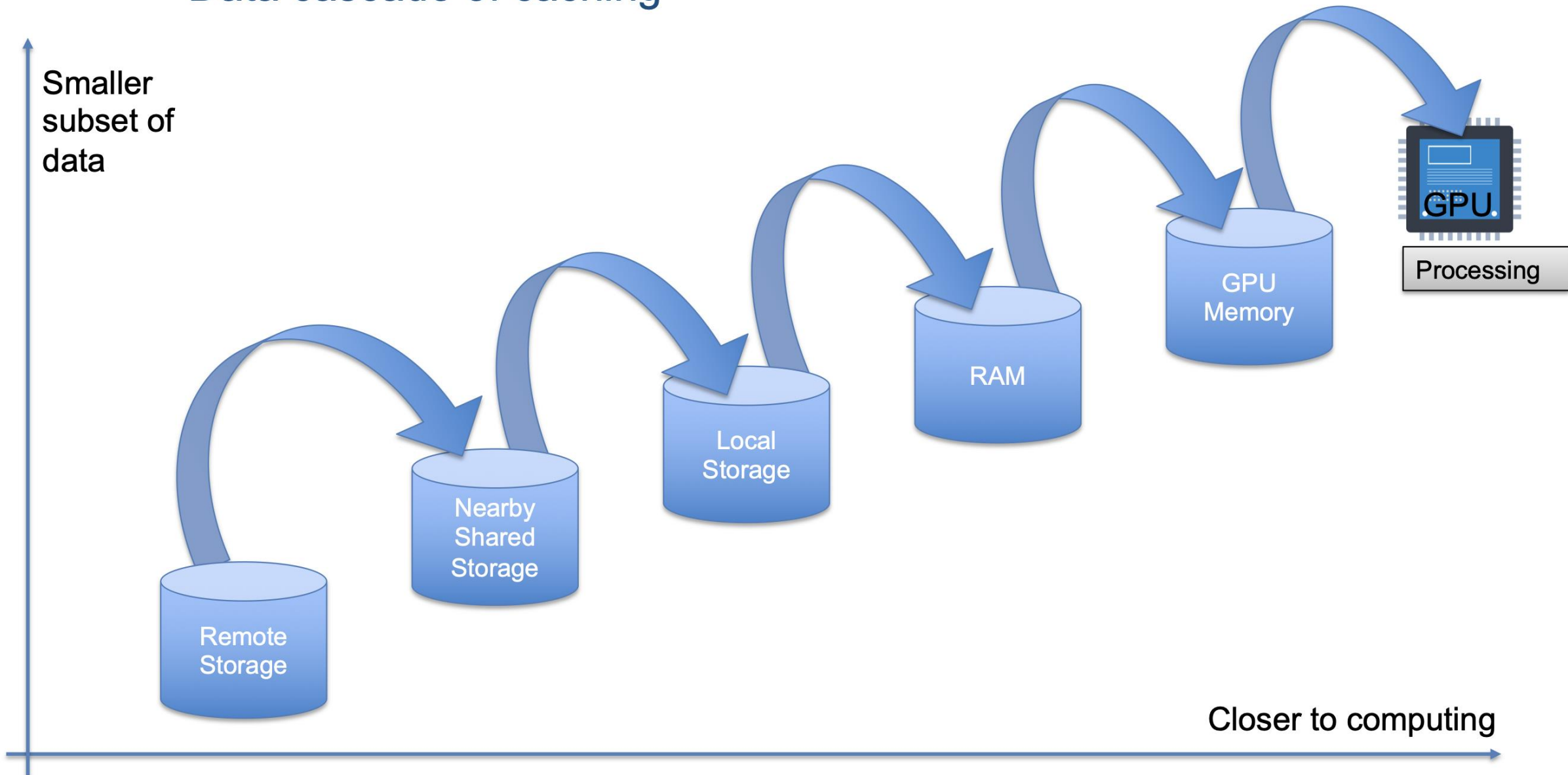


- If you need to read data by timeseries. Store the data by timeseries.
- If you read training samples with 100 parameters for 1 date, do save files containing 100 parameter for 1 dates (but "it depends")
- General solution: create a dataset dedicated to your training task (perform the transposition **offline** if required).

Reformatted datasets



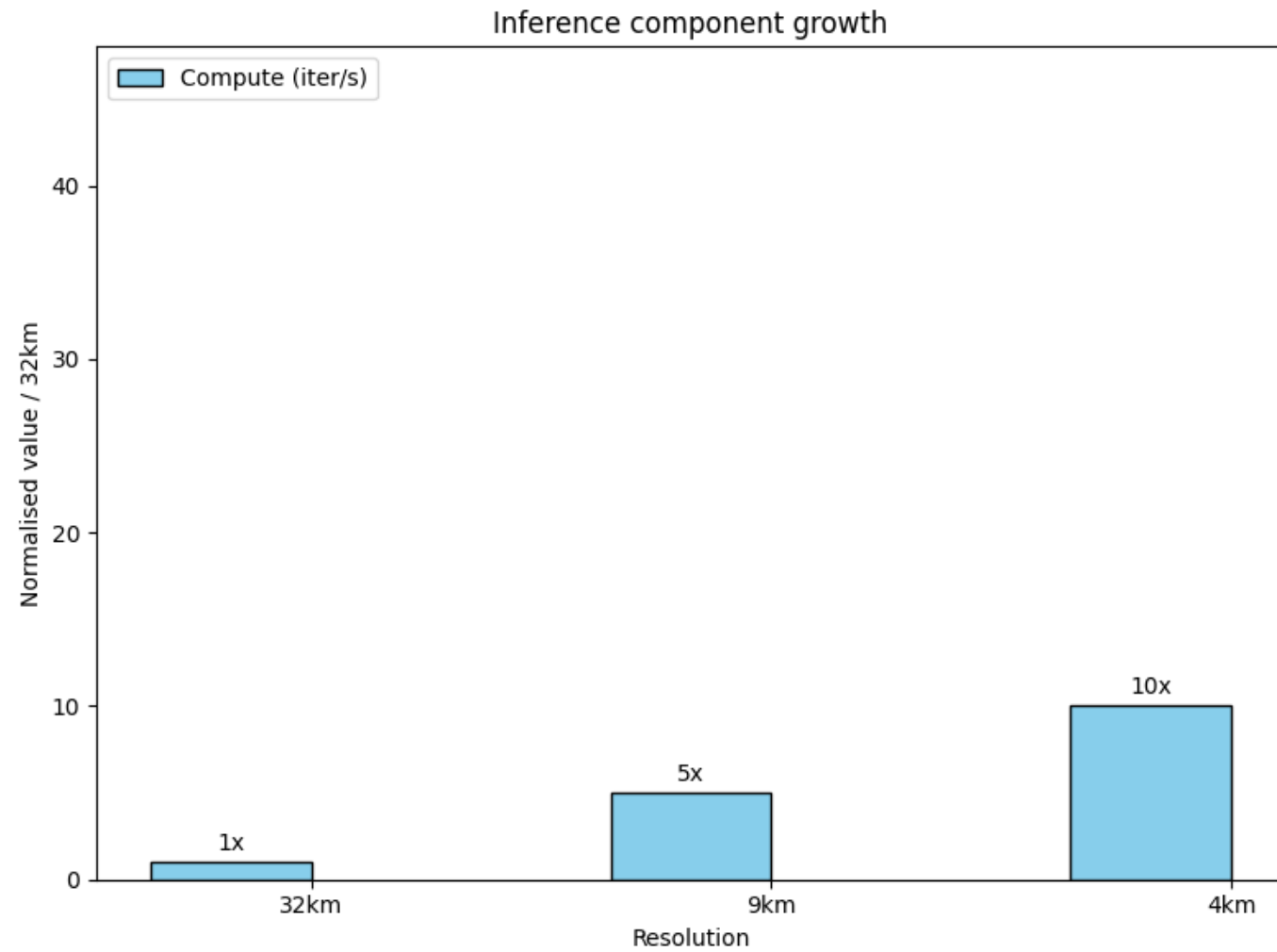
Data cascade of caching



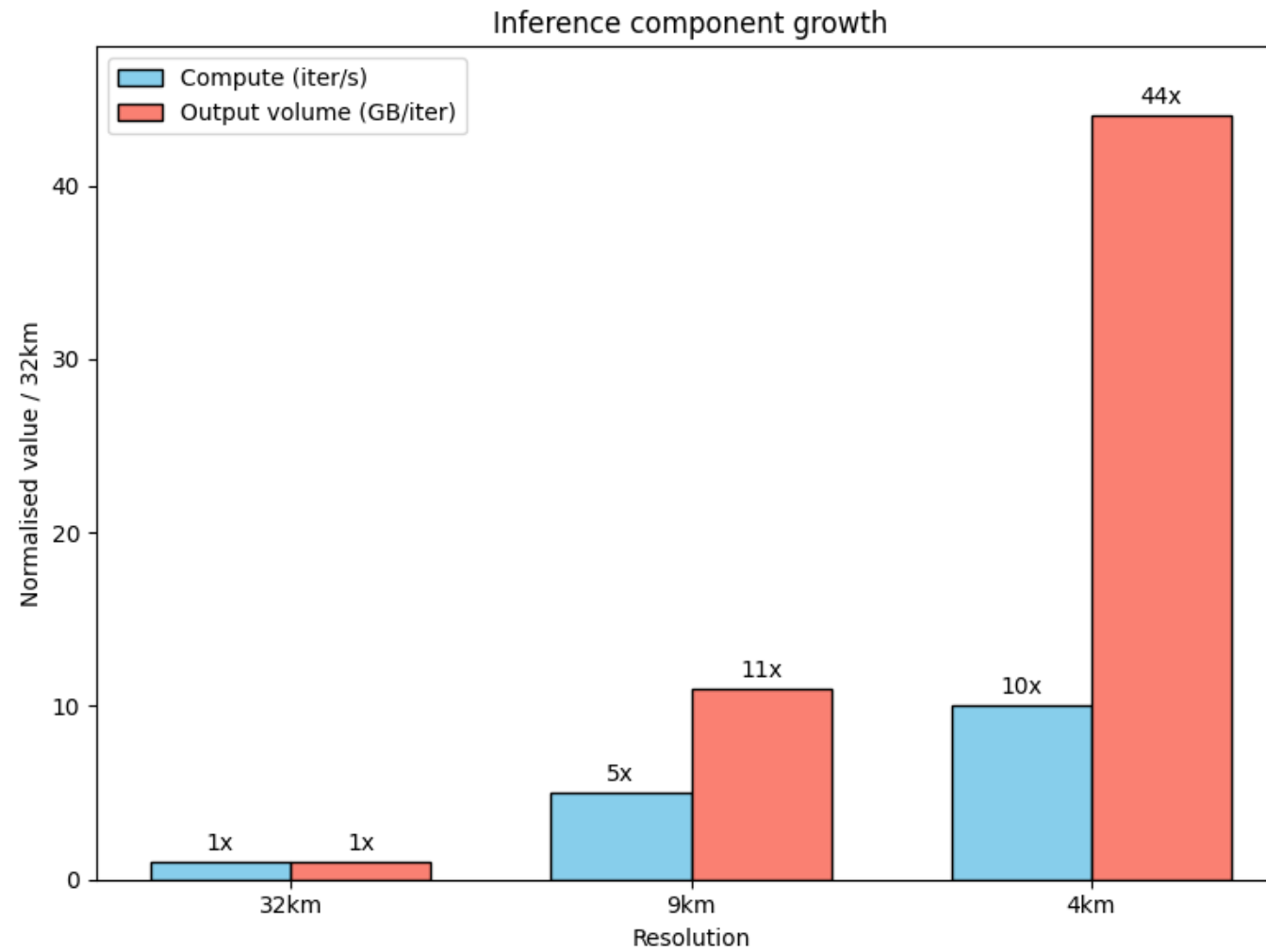
Long-term storage

- SSD
 - Fast
 - Expensive
 - Private
- S3
 - Cheap
 - Limited bandwidth
- Parallel file system (e.g. lustre)
 - Fast
 - Shared
 - Only available on HPC clusters

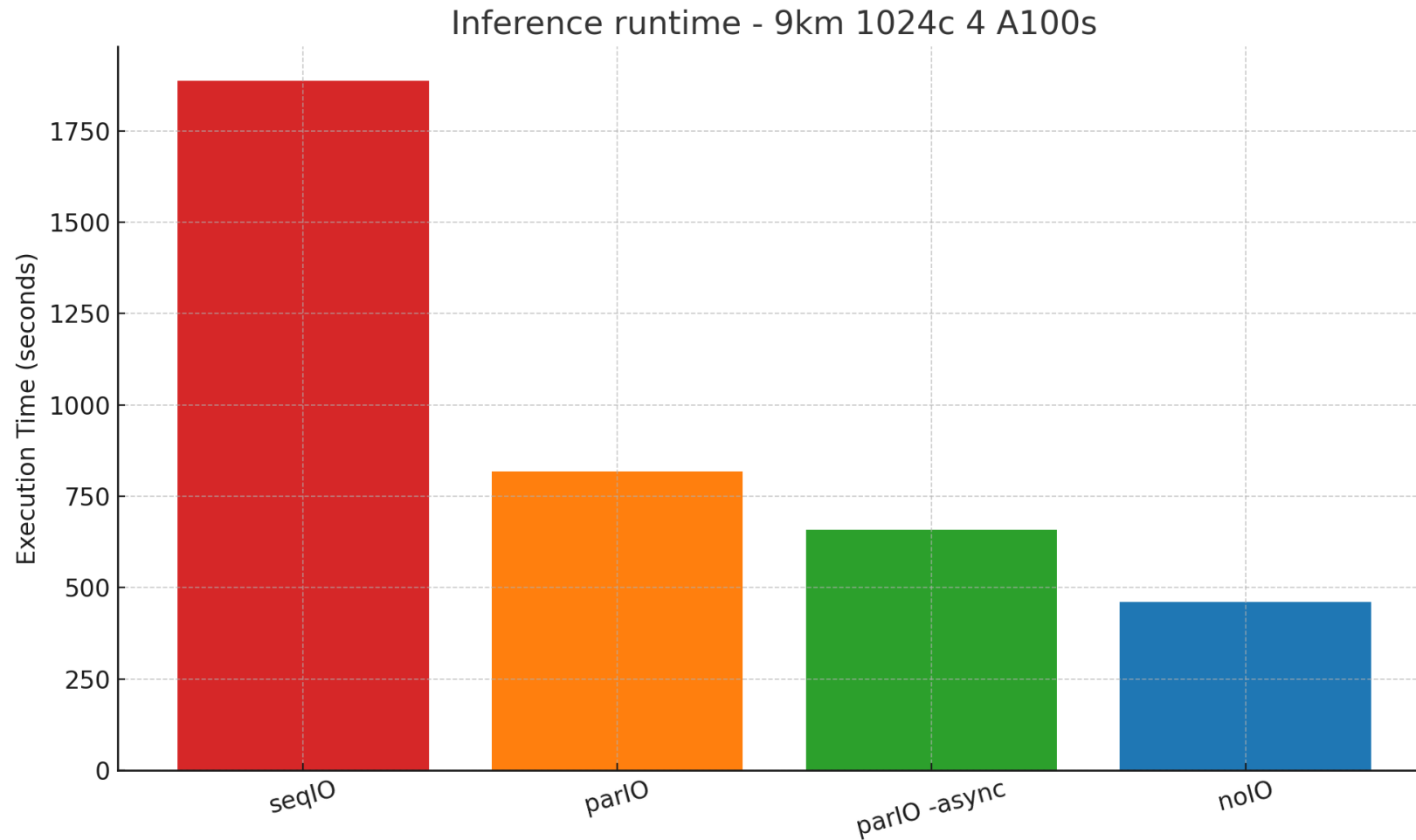
Inference



Inference



Inference - parallel output



Further reading

DATA MOVEMENT IS ALL YOU NEED: A CASE STUDY ON OPTIMIZING TRANSFORMERS

Andrei Ivanov^{*1} Nikoli Dryden^{*1} Tal Ben-Nun¹ Shigang Li¹ Torsten Hoefer¹



GLENN K. LOCKWOOD

SEARCH

Writing to make sense of HPC, storage, and system design

HPC · STORAGE · PERSONAL · ABOUT THIS BLOG · MY WEBSITE

Labels

ai
hpc

February 01, 2025

LLM TRAINING WITHOUT A PARALLEL FILE SYSTEM

FLASHATTENTION-2: Faster Attention with Better Parallelism and Work Partitioning

Tri Dao^{1,2}

But what does this all mean for you?

- Most of the complexity of managing memory is handled by PyTorch or Anemoi
- Try to fill your GPU memory as much as possible
 - Increase batch size, #channels, parameter count
- Play around with torch compile
- Try to fill your CPU memory as much as possible
 - Play with the num_workers parameter
- Think about where your data is coming from
 - Downloading from the internet vs loading from an SSD vs loading from a high performance file system

Intro to parallel training

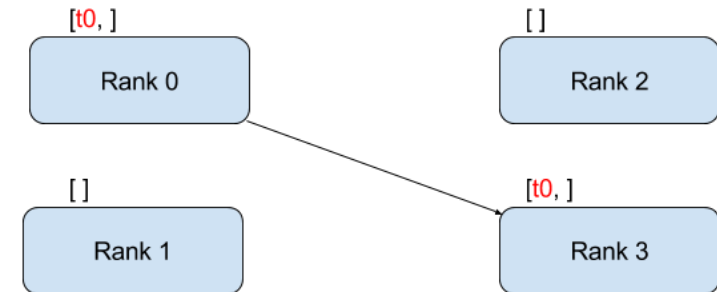
Cathal O'Brien



Parallelism

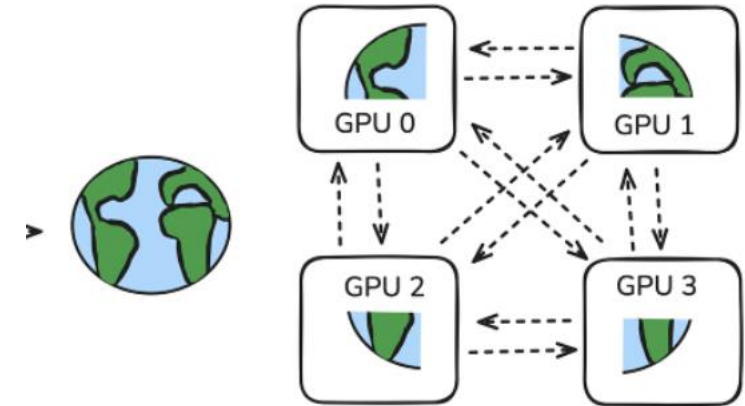
- Train across multiple GPUs at once
 - Each with separate memory
 - GPUs communicate by sending messages
 - Built on top of “torch.distributed” library

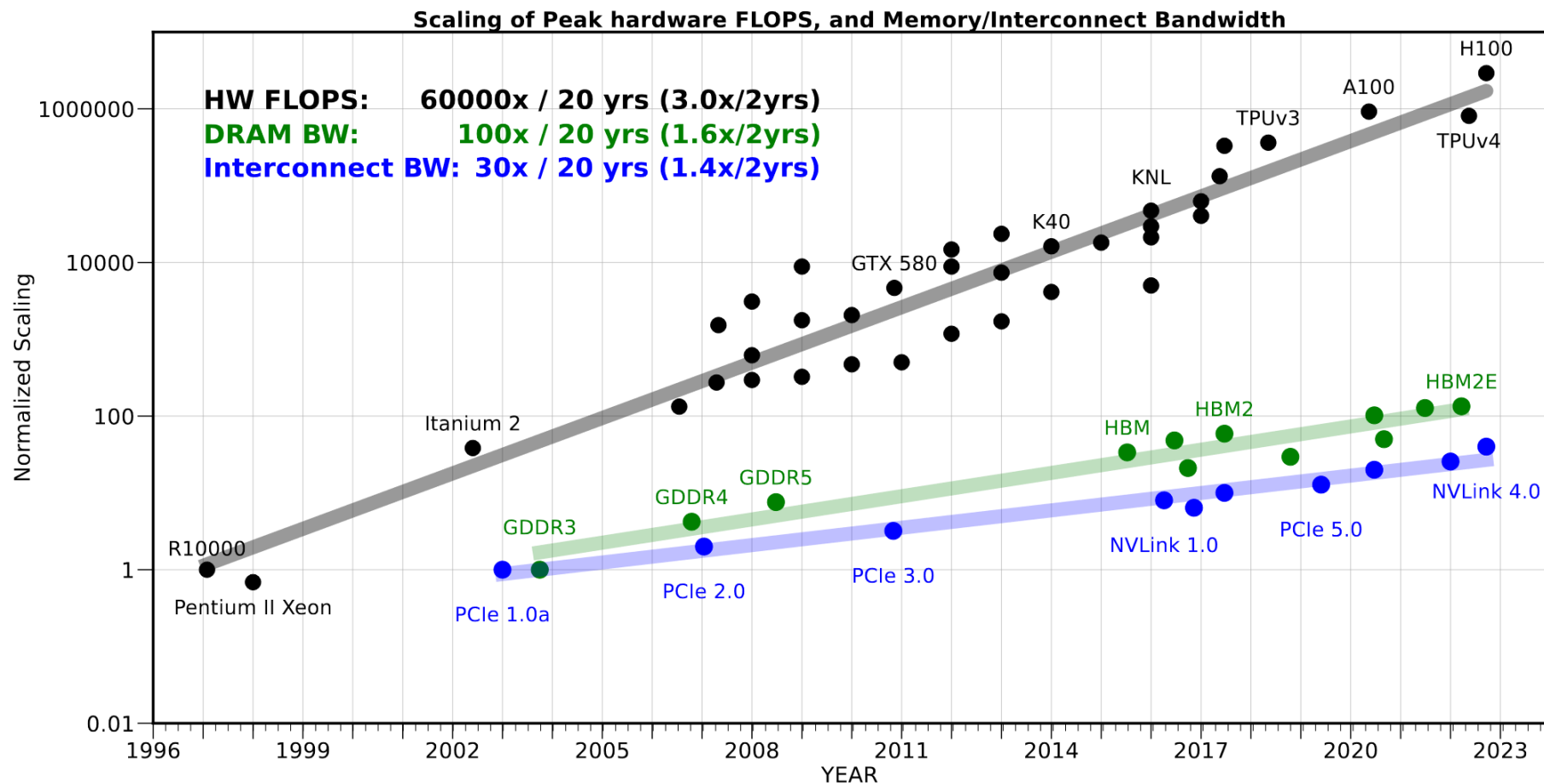
```
"""Blocking point-to-point communication."""  
  
def run(rank, size):  
    tensor = torch.zeros(1)  
    if rank == 0:  
        tensor += 1  
        # Send the tensor to process 1  
        dist.send(tensor=tensor, dst=1)  
    else:  
        # Receive tensor from process 0  
        dist.recv(tensor=tensor, src=0)  
    print('Rank ', rank, ' has data ', tensor[0])
```



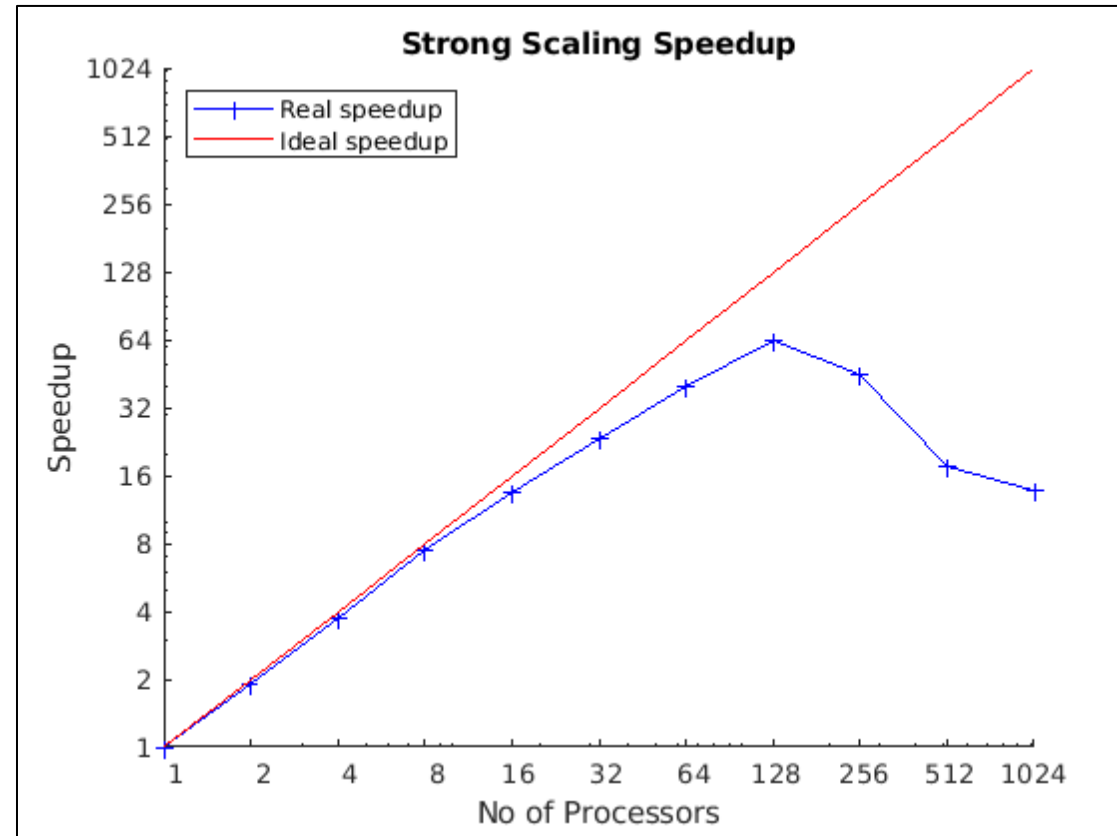
Why do we parallelise?

- Faster training!
 - Share the work across multiple GPUs e.g. 1 attention head per GPU
 - Each can compute a fraction in parallel
 - ... and synchronise their results via messages at the end
- Larger models!
 - Split the globe across multiple GPUs
 - Each GPU only needs to communicate the boundary conditions
 - Never materialise the full globe in a single GPU's memory





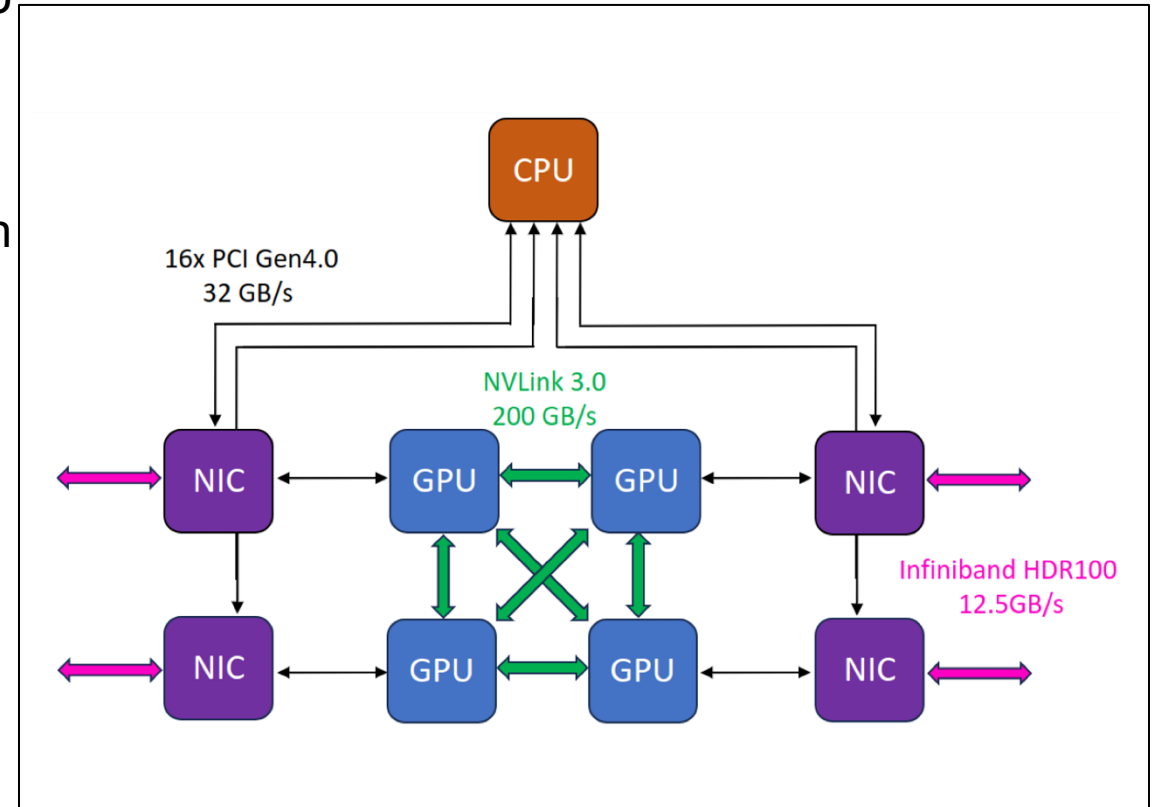
How to train in parallel efficiently



- Communicate as little as possible
- “Hide” communication by overlapping with computation

Network topology

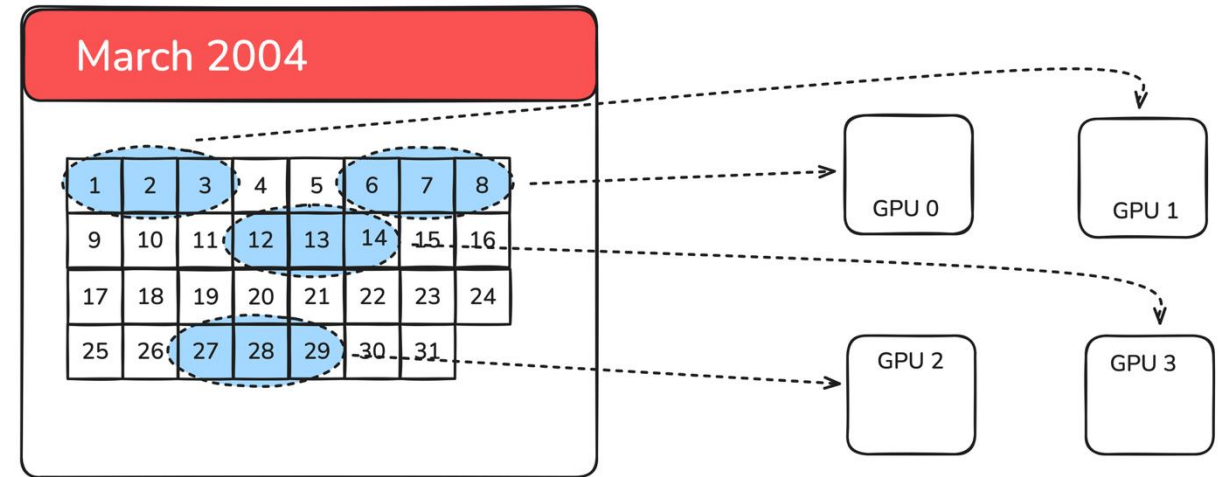
- A node typically has 4 GPUs and 1 CPU
- MUCH faster for GPUs within a node to talk to each other, then to talk to GPUs on other nodes
- => can we communicate more frequently inside a node and less outside a node?



[Leonardo supercomputer](#)

Parallelism in Anemoi

- Data Parallelism: DistributedDataParallel (PyTorch)
 - Distribute training batch across model replicas
 - Aggregate gradients via all-reduce, good scaling 😊
 - Limited by batch size 😞



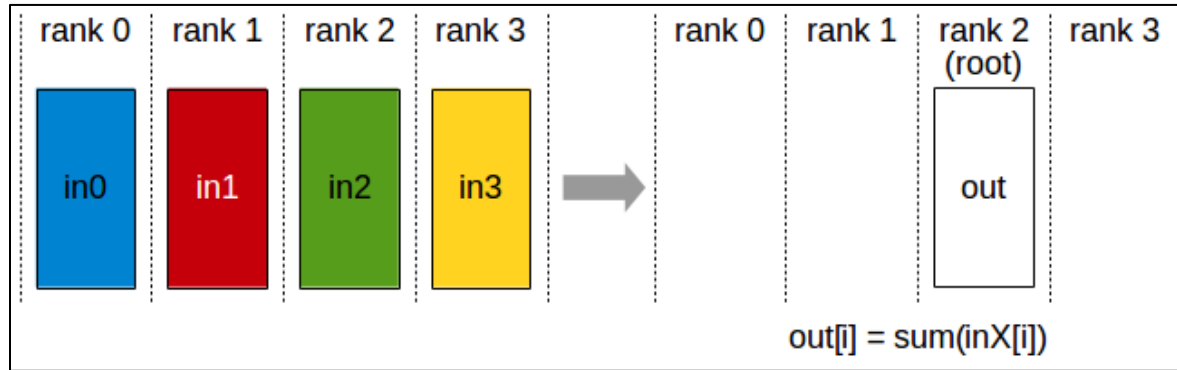
Data parallelism - Naive implementation

```
import torch
import torch.distributed as dist
# initialize the communication group
# define dataloader, loss_fun, and optimizer before starting the training
for inp, label in dataloader:
    optimizer.zero_grad()
    out = model(inp)
    loss = loss_fun(out, label)
    loss.backward()
    # --start--
    for param in model.parameters():
        dist.all_reduce(param.grad.data, op=dist.reduce_op.SUM)
        param.grad.data /= dist.get_world_size()
    # --end--
    optimizer.step()
```

<https://medium.com/@eeyuhao/pytorch-distributed-a-bottom-up-perspective-e3159ee2c2e7>

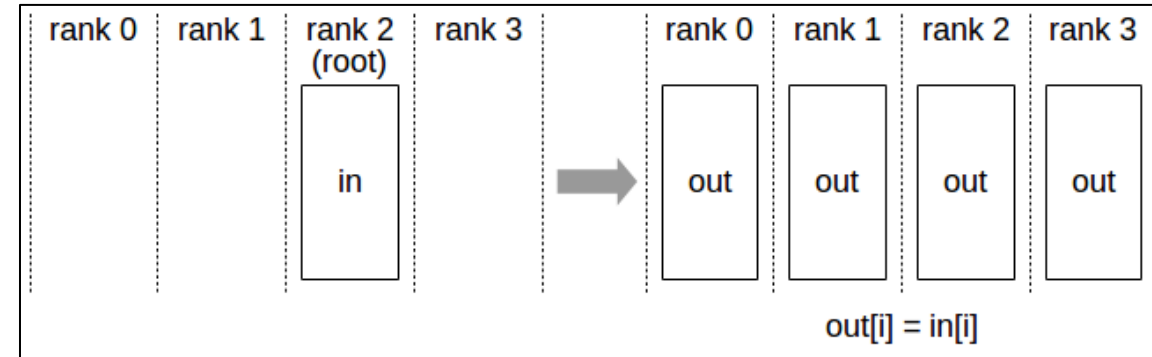
All-reduce

reduce



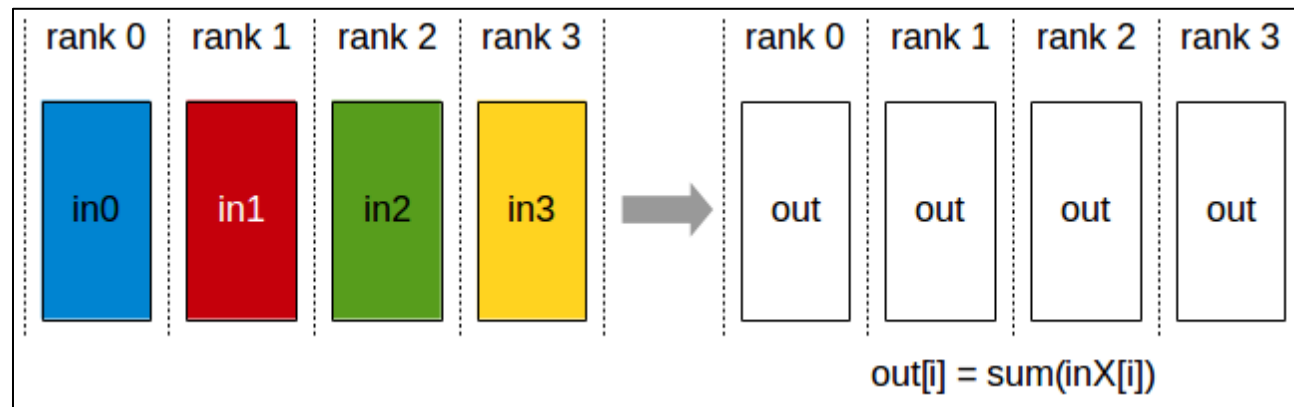
broadcast

+



=

All-reduce



Data parallelism in anemoi

- Implemented using torch.DDP
 - Very efficient
 - Sync parameters during BW pass while computing the next gradients
 - Extremely high overlap rate

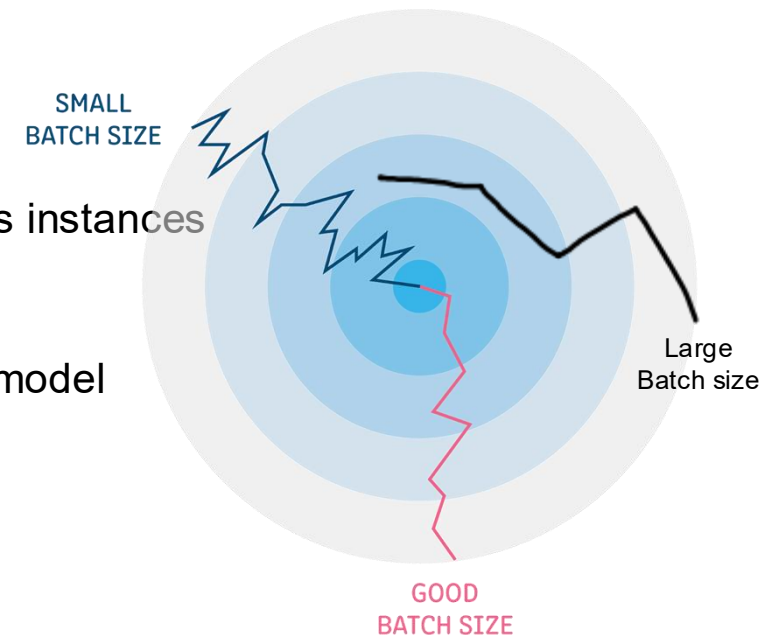
hardware:

num_nodes: 1

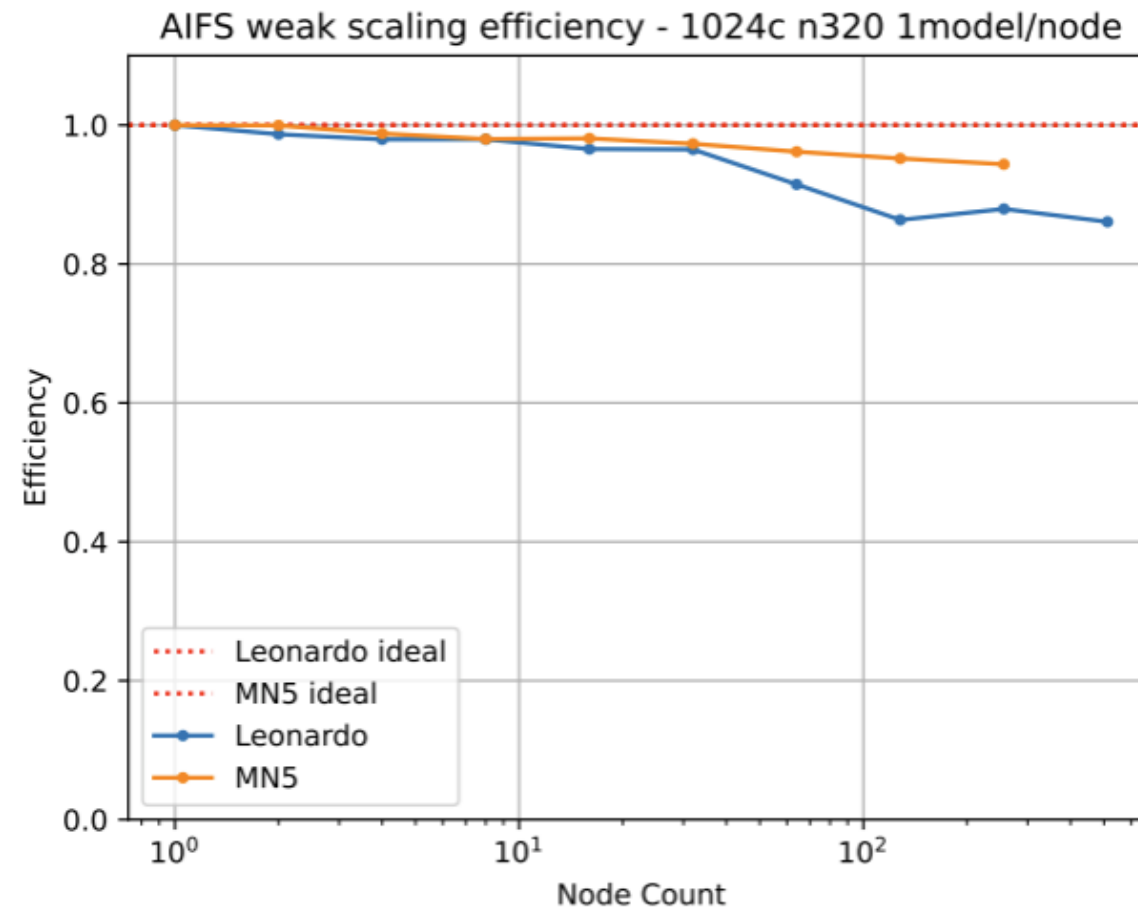
num_gpus_per_node: 4

num_gpus_per_model: 1

- Using DDP changes your result!
 - Aggregate batch size = batch size per model * number of models instances
 - A batch size too large and your convergence can suffer
 - For AIFS training, we used a batch size of 16 (1 per model * 16 model instances)



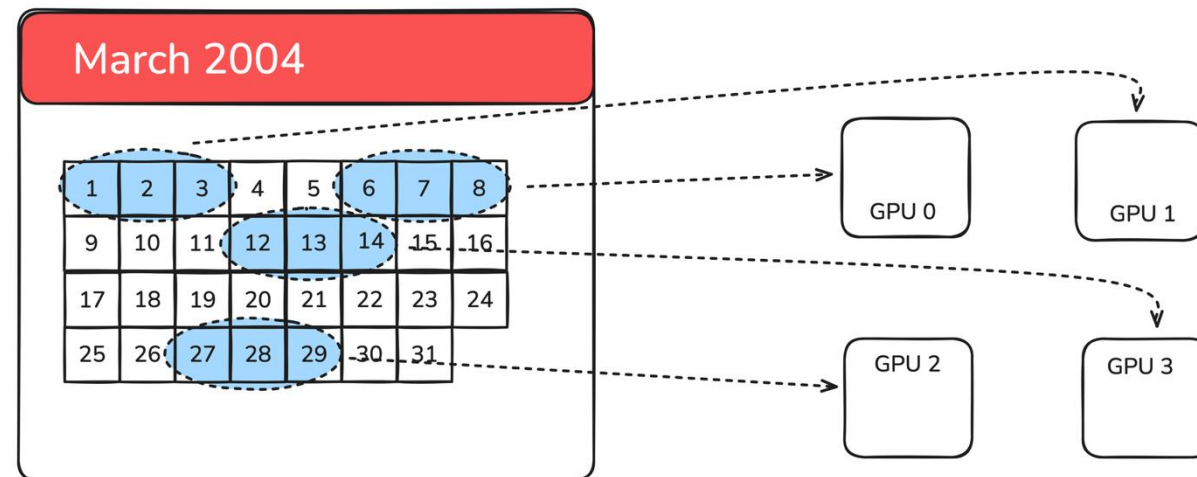
Data parallelism - scaling



Parallelism in Anemoi

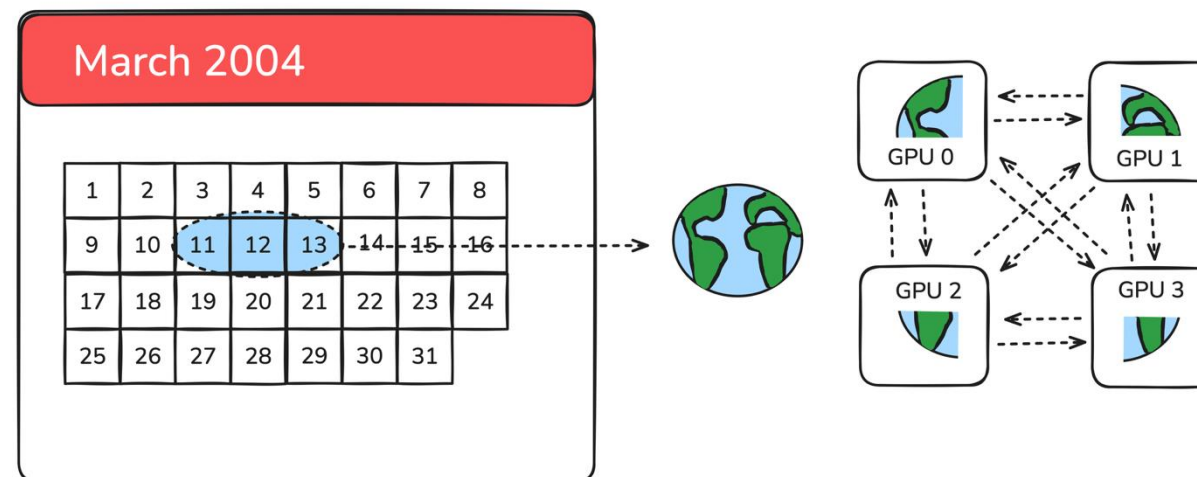
- Data Parallelism: DistributedDataParallel (PyTorch)

- Distribute training batch across model replicas
- Aggregate gradients via all-reduce, good scaling 😊
- Limited by batch size 😞



- Model parallelism: domain-specific sharding

- Distribute input data and activations across GPUs
- Collective communication to handle synchronisation
- Limited by communication overheads



Model parallelism

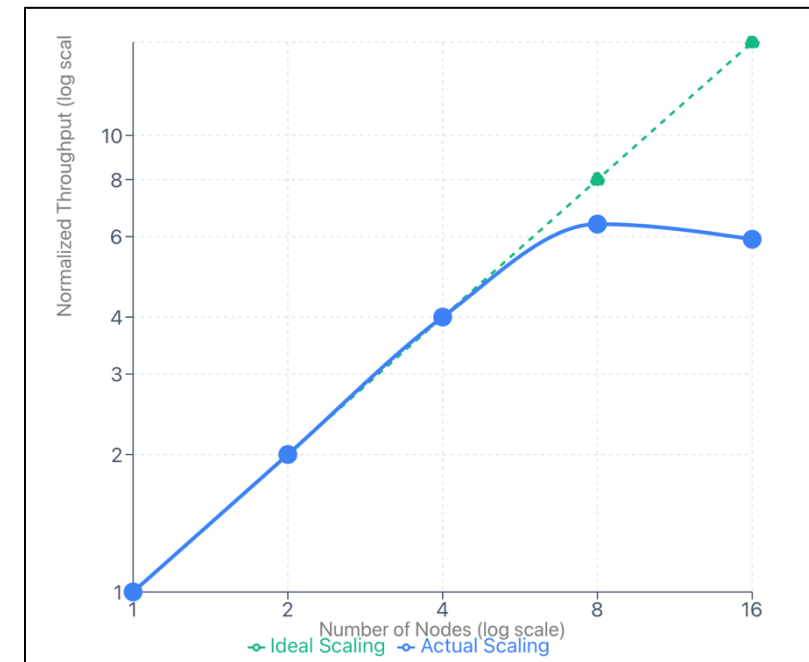
- Can increase your throughput greatly
 - Output is the same
- Communication heavy
 - Collective communication
 - Not overlapped
- Does not scale perfectly
 - Trade-off between efficiency and Wallclock time
 - Right answer depends on how busy your cluster is. More nodes = more queuing!

hardware:

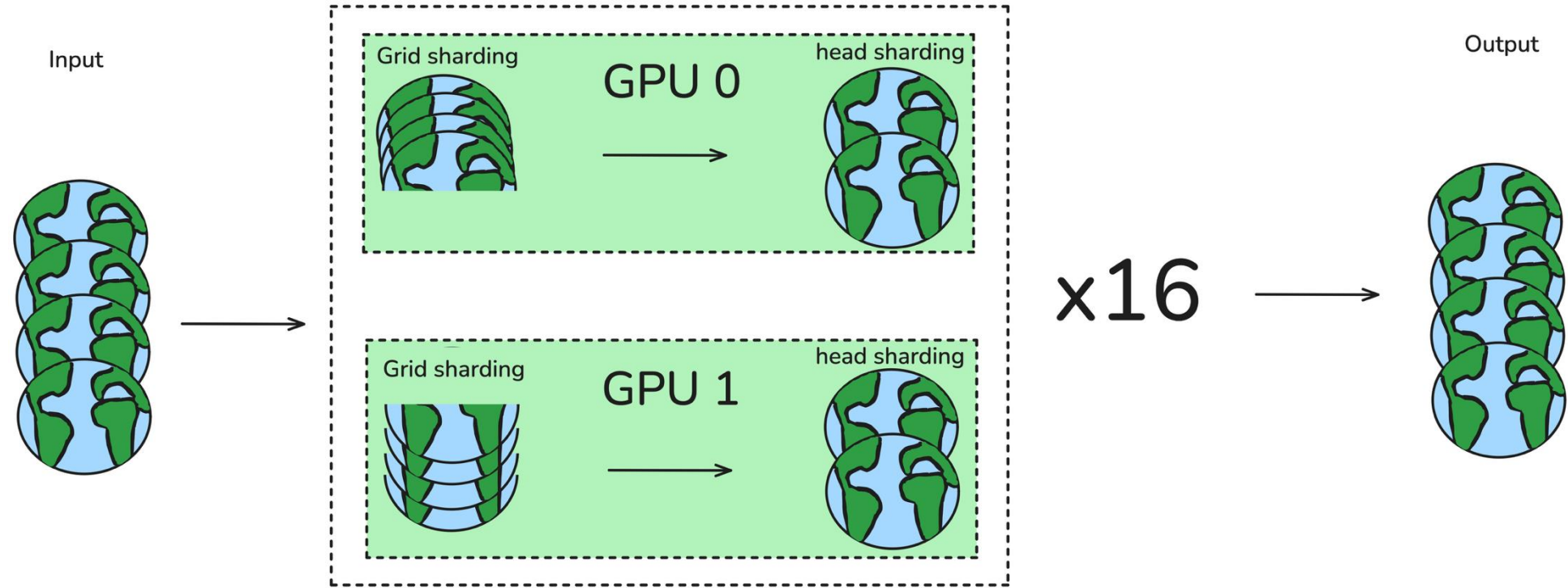
num_nodes: 1

num_gpus_per_node: 4

num_gpus_per_model: 4

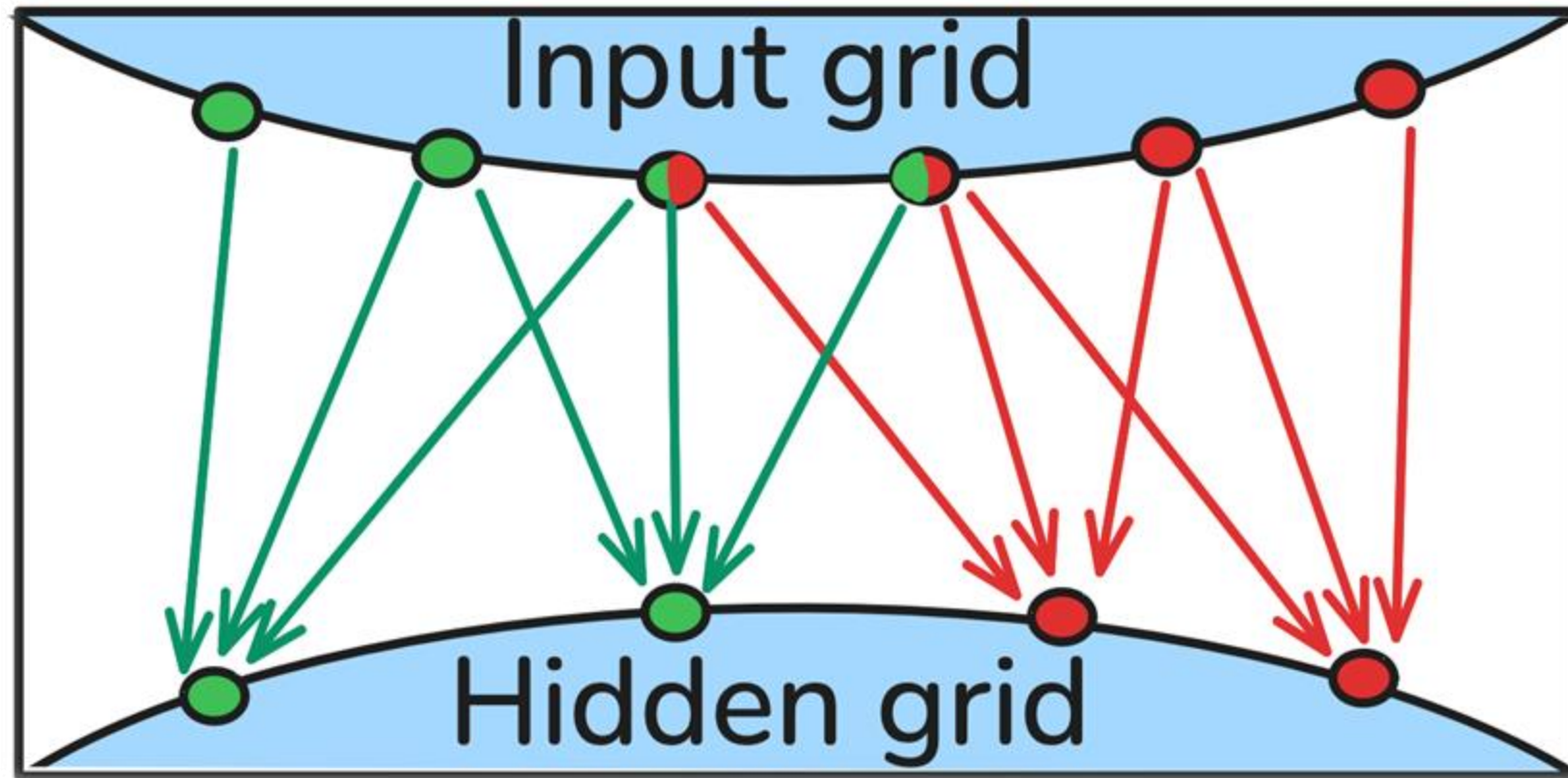


Sharding Attention



- 4x all-to-all communication per attention head (1 * encoder + 16 * processor + 1 * decoder)
- But: we can do better by exploiting sparsity + locality of graph attention

Mapper edge sharding

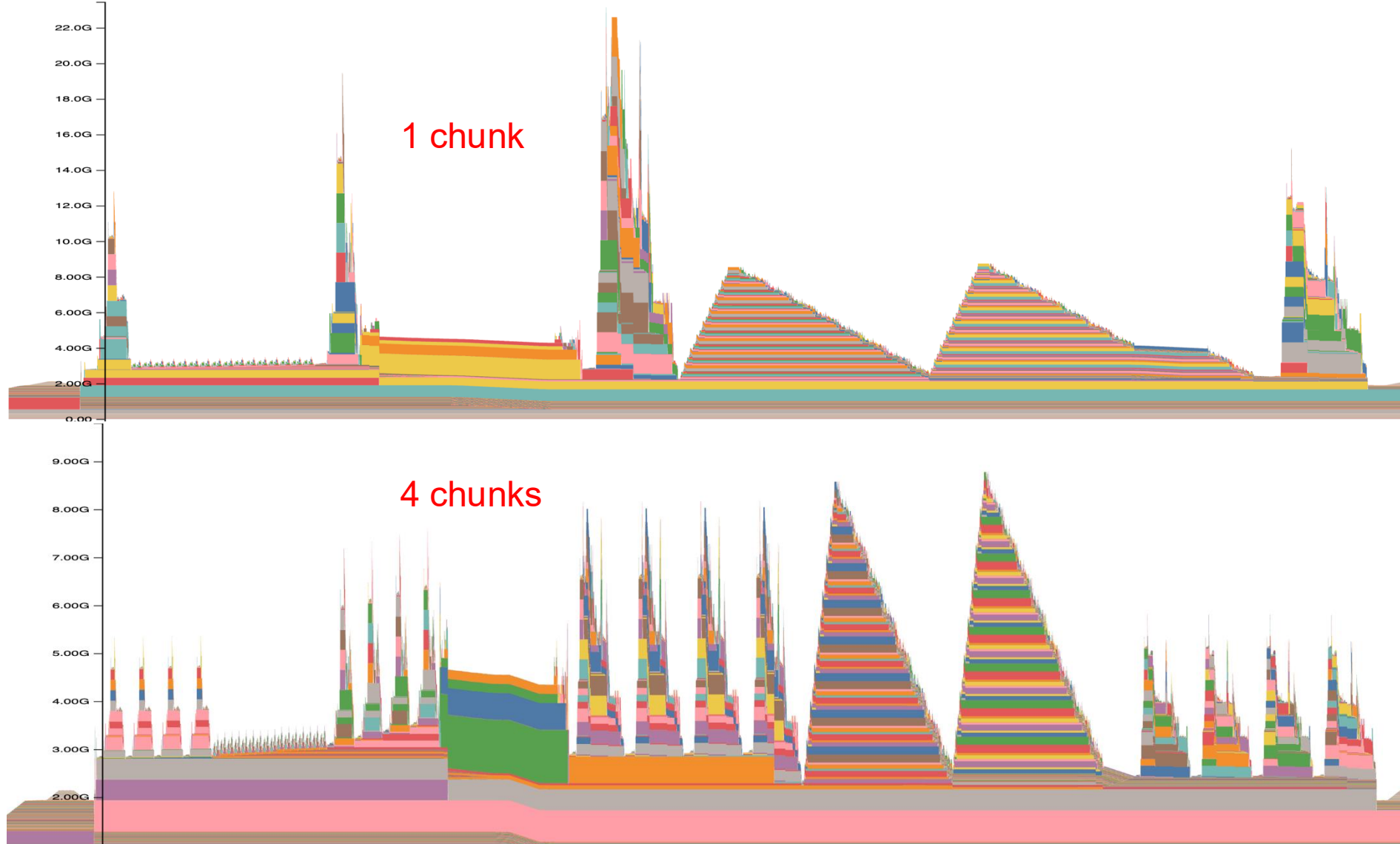


- 1x all-gather in fwd + reduce-scatter in bwd
- + independent subgraphs on each device (with small overlap)

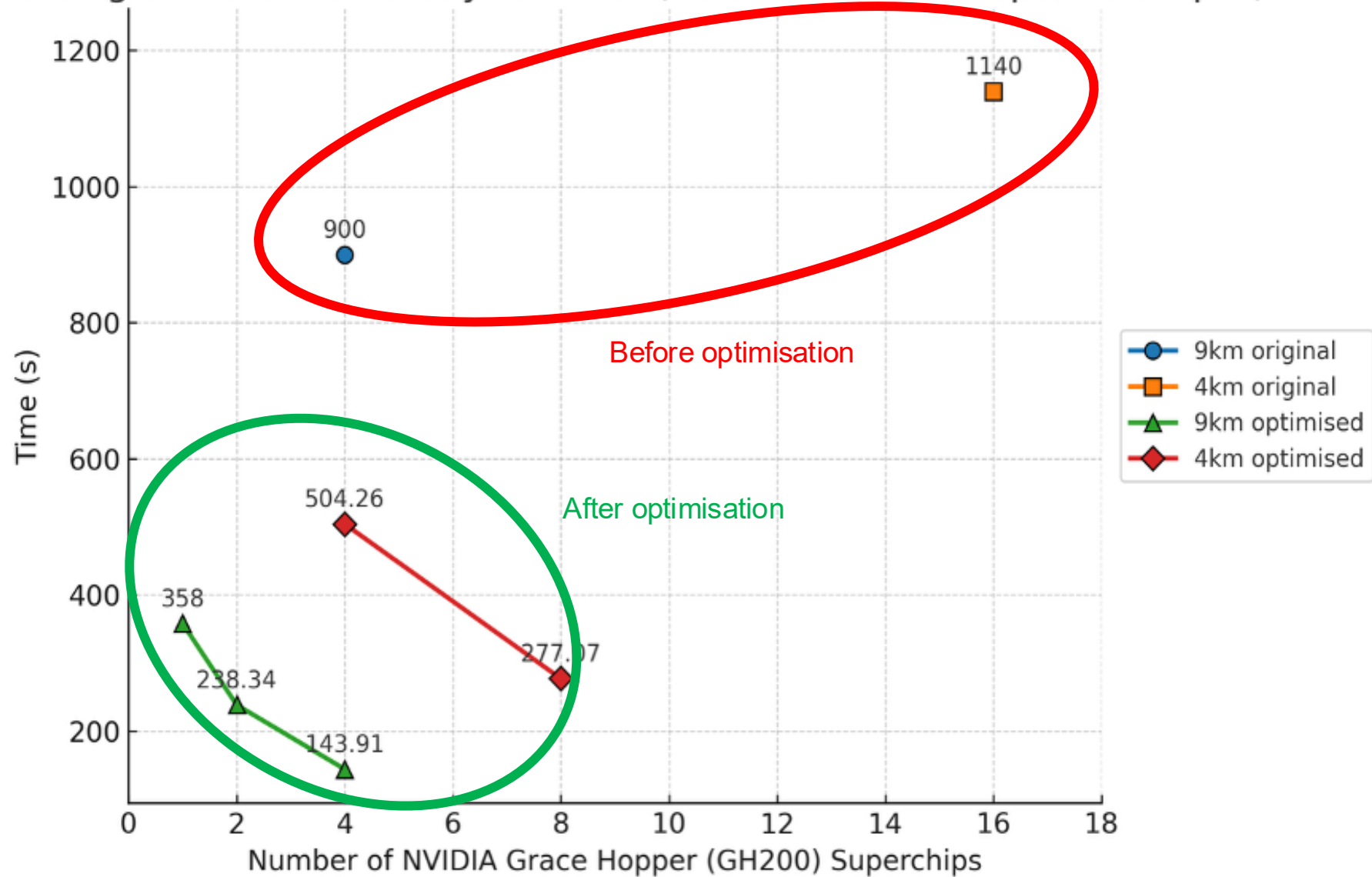
Mapper chunking

- Running inside 1 GPU now
 - but we are breaking our subgraph into smaller subgraphs to reduce peak mem

Memory
usage



AIFS single inference 15-day forecast (100 fields, 150 steps, no output)



Further reading

AERIS: Argonne Earth Systems Model for Reliable and Skillful Predictions

Väinö Hatanpää^{1,+}, Eugene Ku^{1,+}, Jason Stock^{1,+}, Murali Emani¹, Sam Foreman¹, Chunyong Jung¹,
Sandeep Madireddy¹, Tung Nguyen⁴, Varuni Sastry¹, Ray A. O. Sinurat², Sam Wheeler¹,
Huihuo Zheng¹, Troy Arcomano^{1,3}, Venkatram Vishwanath^{1,*}, and Rao Kotamarthi^{1,*}

¹Argonne National Laboratory, Lemont, Illinois, USA

²University of Chicago, Chicago, Illinois, USA

³Allen Institute for AI, Seattle, Washington, USA

⁴University of California, Los Angeles, California, USA

⁺*Joint First Authors* ^{*}{venkat,vrkotamarthi}@anl.gov

- Gordon Bell submission
 - Adds further parallelisms: pipeline and window parallelism
 - Scaled across 10,000 nodes
 - Didn't converge!

Take-aways

- Think about network topology
 - Model parallelism inside a node
 - Data parallelism between nodes
- Use data parallelism freely
 - But beware a too large batch!
- Use model parallelism
 - But not too much or you might queue for very long

Extras

- Parallel inference also possible:
 - <https://anemoui.readthedocs.io/projects/inference/en/latest/inference/parallel.html>